May 2026

Query-based Dependent Type Elaborator

Submitted in partial fulfilment of the requirements for the Computer Science Tripos, Part II

Declaration of originality

I, the candidate for Part II of the Computer Science Tripos with Blind Grading Number 7268C, hereby declare that this report and the work described in it are my own work, unaided except as may be specified below, and that the report does not contain material that has already been used to any substantial extent for a comparable purpose. In preparation of this report, I adhered to the Department of Computer Science and Technology AI Policy. I am content for my report to be made available to the students and staff of the University.

Date: *May 2026*

Proforma

Candidate Number	7268C
Title	Query-based Dependent Type Elaborator
Examination	Computer Science Tripos Part II, 2026
Word Count	11207[1]
Code Line Count	14,149[2]
Project Originator	The candidate
Project Supervisor	Dr Jon Sterling
Ethics Approval	Not required

Original aims

Dependently typed languages such as Lean, Agda, and Rocq let types depend on terms, so type checking must execute *arbitrary programs* to compare them. A change to a function's body can therefore affect downstream type checks even when its type is unchanged. Existing type checkers re-elaborate the suffix of the file from the changed name, plus every file that imports it, even when no use actually depends on the change.

This project set out to implement an elaborator for a dependently typed language, and to investigate whether per-declaration queries could re-elaborate only the definitions actually affected by each edit.

Work completed

All success criteria were met and substantially exceeded. The elaborator type-checks an expressive dependent type theory, capable of elaborating code from the Lean 2 HoTT library. Incremental re-elaboration takes time proportional to the edit, independent of project size. Beyond the proposal, a machine-checked correctness proof for the build system underlying the elaborator was established in Lean 4 against a reference batch semantics, extending without modification to effectful variants.

Special difficulties

None.

[1] Via the `wordometer` Typst package.

[2] Via `cloc`, computed by a handwritten script.

Contents

1 Introduction

Interactive proof assistants are live programming environments for mathematics. The user edits a definition, and the elaborator type-checks and reports diagnostics. In a library at the scale of mathlib, a single edit forces re-elaboration of every file that imports the changed one, and the cost is paid on every save. Reducing that cost without trusting an unverified cache is the question this thesis addresses.

1.1 Motivation

Lean's mathlib library [The mathlib Community, 2020] contains over 130,000 definitions and 270,000 theorems.[3] Most of the re-elaboration triggered by an edit is wasted: mainstream proof assistants track dependencies at the granularity of module imports, so an edit invalidates everything its file transitively imports, even when no declaration in that closure actually unfolded the change. A finer scheme would record, for each declaration, the constants its elaboration actually unfolded; this set cannot be precomputed from the syntax, because a dependent type theory's *conversion rule* decides at runtime which earlier definitions to unfold. Existing systems approximate at varying granularities: Wenzel's Isabelle/PIDE [Wenzel, 2014] and Lean's snapshot trees [Ullrich, 2023] go to command-suffix within a file; Sixty [Fredriksson, 2019] goes to per-declaration in memory within a language-server session.

Whatever the granularity, the incremental cache must agree with batch elaboration: a cache hit returning a value the elaborator's source code would not have produced is undetectable from interaction alone. Precise invalidation requires recording the cross-declaration dependencies a conversion check discovers at runtime, which extends the trusted layer to include that record. Correctness requires that a cache hit return what a fresh build would. No deployed system has met both.

1.2 Aims

The core aims of this project correspond to the two success criteria of the proposal: correctly type-checking a dependent type theory, and showing measurable incremental performance improvement against re-elaboration. Specifically, we seek

Core aims

(a) to build an **elaborator** for a dependent type theory with Π types, inductive types, universe polymorphism, structures, demonstrated on subsets of the Lean 2 HoTT library (Section 4.1), and

(b) to achieve **substantial speedup** of cached rebuilds against re-elaboration across a range of edit categories (Section 4.3).

[3] https://leanprover-community.github.io/mathlib_stats.html, as of 2026-05-14.

Extension aims

(a) to give **formally specified guarantees** about our underlying incremental system, in order to remove this aspect of our elaborator from the *trusted computing base*, in particular, to show that our elaborator is semantically identical to *batch elaboration* (Section 3.2, Section 3.3), and

(b) to **modularise** our code in such a way that extending our incremental system is *easy* and can support the necessary features expected of modern tooling, such as *profiling*, *tracing*, and *language servers*. (Section 3.3.4).

2 Preparation

Given that our core aims are to produce a novel, incremental elaborator, it is appropriate for us to first study the state-of-the-art algorithms in this area. Hence, we first explain the type theory used by our language in Section 2.1, and also introduce the *conversion rule*. In Section 2.2 and Section 2.3, we introduce the *bidirectional type-checking* discipline and analyse the algorithm of *normalisation by evaluation*.

In Section 2.4, we present the vocabulary of *build systems*. Section 2.5 reviews the norms of proof assistants and incrementalisation in adjacent settings. Finally, we analyse the project's requirements in Section 2.6 and outline the software engineering practices in Section 2.7.

2.1 Our dependent type theory

The elaborator's internal language is a version of dependent type theory in the tradition of [Martin-Löf, 1984]. The distinguishing form is the *dependent function type* $\Pi_{x:A}B$, whose introduction and elimination rules are:

$$\frac{\Gamma, x : A \vdash b : B}{\Gamma \vdash \lambda x.b : \Pi_{x:A}B}\,\Pi\text{-INTRO} \qquad \frac{\Gamma \vdash f : \Pi_{x:A}B \quad \Gamma \vdash a : A}{\Gamma \vdash f\ a : B[a/x]}\,\Pi\text{-ELIM}$$

The elimination rule $B[a/x]$ is what distinguishes a dependent type theory from a simple one, as the resulting type is allowed to *depend* on the argument, $a : A$.

2.1.1 Grammar

The calculus uses universes *à la Tarski* [Martin-Löf, 1984][4], which separates the grammar into *mutually* defined terms and types:

	Grammar	Surface syntax[5]	Description
$\ell ::=$	$\omega \mid 0 \mid \mathrm{succ}(\ell) \mid \max(\ell, \ell)$	`0, u + 1, max u v`	universe levels
$A, B ::=$	Type_ℓ	`Type u`	universe
$\mid$	$\Pi_{x:A}B$	`(x : A) → B`	dependent function type
$\mid$	$\mathsf{El}(t)$	*(no syntax)*	decoding a type code
$t, s ::=$	x_i	`x`	variable
$\mid$	$c.\{\overline{\ell}\}$	`c.{u, v}`	constant
$\mid$	$\lambda x : A.t$	`fun x : A ⇒ t`	function abstraction
$\mid$	$t\ s$	`t s`	application
$\mid$	t_i	*(no syntax)*	field projection
$\mid$	$\mathsf{let}\ x : A := t\ \mathsf{in}\ s$	`let x : A := t; s`	let-binding
$\mid$	Type'_ℓ	*(no syntax)*	universe code
$\mid$	$\Pi'_{x:t}s$	*(no syntax)*	function type code

Variables x_i are *de Bruijn indices* [Bruijn, 1972], where i refers to the i-th enclosing binder counted from

[4] The popular *Russell*-style alternative collapses terms and types into one. The Tarski formulation has cleaner categorical semantics, discussion of which is out-of-scope.

[5] We borrow the surface syntax of Lean 4.

the use site.[6] Global constants $c.\{\overline{\ell}\}$ carry a list of universe arguments, and $t.k$ projects the k-th field. The primed term formers Type'_ℓ and $\Pi'_{x:t}s$ are term-level *codes* for the corresponding type formers; the type-level constructor $\mathsf{El}(t)$ decodes a code into a type. Inductive declarations introduce a new type former, its constructors, and a recursor.

2.1.2 Universes

Types are stratified into a hierarchy

$$\mathsf{Type}_0 : \mathsf{Type}_1 : \mathsf{Type}_2 : \ldots$$

with $\mathsf{Type}_u : \mathsf{Type}_{u+1}$ for each u. The hierarchy is *non-cumulative*: each Type_u inhabits exactly one universe, Type_{u+1}.

A definition can be parameterised by universe levels:

```
def id.{u} (α : Type u) (x : α) : α := x
```

Levels are a separate sort of variable from term-level binders and must be instantiated explicitly at each use site, e.g. `id.{1}`. There is no universe-level inference: the programmer writes every level argument.[7]

2.1.3 Inductive types

An inductive type [Dybjer, 1994] is introduced by its constructors. Consider the following declaration of natural numbers:

```
inductive Nat : Type where
  | zero : Nat
  | succ (n : Nat) : Nat
```

This generates the constants `Nat : Type`, `Nat.zero : Nat`, and `Nat.succ : Nat → Nat`. In addition, a recursor, which encodes the *dependent elimination principle*[8], is generated:

```
axiom Nat.rec.{u} :
  (motive : Nat → Type u) →
  (motive Nat.zero) →
  ((n : Nat) → motive n → motive (Nat.succ n)) →
  (n : Nat) → motive n
```

This eliminator is *dependent*: the result type may vary with the scrutinee, so the recursor takes a *motive* — a function from the inductive type to a type — that picks out the result type at each value. It then takes one *branch* per constructor. The branch's parameters are the constructor's own arguments together with one *inductive hypothesis* per recursive argument, the motive evaluated at that sub-term.

```
def Nat.add (m n : Nat) : Nat :=
  Nat.rec
    (fun n ⇒ Nat)            -- Regardless of n, our result type is Nat
    m                        -- n = Nat.zero; return m
```

[6] However, we still retain binder names at the *binding* site, for the purposes of pretty-printing.

[7] Inference is also possible, which would require a constraint solver over this particular join-semilattice.

[8] Eliminators are sufficient to define all structurally recursive functions on the type. We avoid implementing *pattern matching* for its complexity.

```
    (fun n ih ⇒ Nat.succ ih) -- n = Nat.succ k; assume ih = Nat.add m k
    n
```

This encodes the two defining equations:

```
Nat.add m Nat.zero     = m
Nat.add m (Nat.succ n) = Nat.succ (Nat.add m n)
```

2.1.4 Reduction and conversion

To equate terms that compute to the same value, three notions are distinguished. *Reduction* is a directed rewrite $t \rightsquigarrow s$, generated by five rules each named by a Greek letter. *Definitional equality* ($t \equiv s$) is the smallest congruence containing reduction in either direction. The *conversion rule*

$$\frac{\Gamma \vdash t : A \qquad A \equiv B}{\Gamma \vdash t : B}\,\textsc{Conv}$$

lets a term cross a definitional equality without an explicit coercion; this is what distinguishes dependent type checking from simple type checking. *Conversion checking* is the algorithm that decides definitional equality, which we specify in Section 2.3.

The five rules are:

- β: $(\lambda x.b)\,a \rightsquigarrow b[a/x]$, substitution.
- δ: $c \rightsquigarrow t$, assuming the global environment binds the constant c to body t, definition unfolding.
- ζ: $\mathsf{let}\ x := a\ \mathsf{in}\ b \rightsquigarrow b[a/x]$, let inlining.
- η:
    1. at function types, $f \equiv \lambda x.\, f\ x$
    2. at single-constructor inductives (structures), $s \equiv c(s.1, \ldots, s.k)$.
- ι: a recursor applied to a constructor reduces to the corresponding branch. For Nat,
    1. $\mathsf{Nat.rec}\ C\ z\ s\ \mathsf{zero} \rightsquigarrow z$
    2. $\mathsf{Nat.rec}\ C\ z\ s\ (\mathsf{succ}\ n) \rightsquigarrow s\ n\ (\mathsf{Nat.rec}\ C\ z\ s\ n)$.

As a worked example, the elaborator checks `Eq.refl.{0} Nat 6 : Nat.add 2 4 = 6` by reducing the left-hand side. δ unfolds `Nat.add` to the recursor. Then ι fires as the recursion's principal argument is unwound from `4` through `succ`s down to `0`, and the term reduces to `succ (succ (succ (succ 2)))`, which is `6`.

Of the five rules, only δ crosses declaration boundaries: applying it looks up another constant in the global environment. The other four operate on a term's own structure. A conversion check that fires δ on `foo` depends on `foo`'s body; one that does not is independent of any change to `foo`. The build system tracks these dynamic cross-declaration dependencies (Section 3.2).

2.2 Bidirectional type-checking

The reduction rules of Section 2.1 equip the elaborator to compare two types. Finding the type of a term in the first place is a separate problem. In a dependent type theory the unifier would have to invent terms (since terms appear in types), and the resulting problem, *higher-order unification*, is undecidable [Goldfarb, 1981]. [Pierce and Turner, 2000] set out a different discipline: the programmer

supplies enough annotations that the checker never has to invent a type. [Coquand, 1996] adapted this to dependent types.

Bidirectional checking splits the typing relation into two judgements with opposite directions of information flow:

$$\Gamma \vdash e \Leftarrow A \qquad \textit{check} \text{ if the term } e \text{ has type } A$$
$$\Gamma \vdash e \Rightarrow A \qquad \textit{synthesise} \text{ the unique } A \text{ that types the term } e$$

Every syntactic form has exactly one applicable mode. Introduction forms (λ, structure constructor) are *checkable*: the type of $\lambda x.b$ is fixed by the surrounding context, since the binder's domain has to come from outside. Elimination forms (variables, applications, projections) are *synthesisable*: the head determines the type. [Dunfield and Krishnaswami, 2021] survey the modes and rules in detail.

A checking judgement falls back to synthesis when no checking rule matches. The elaborator synthesises the term's type B, and the conversion checker decides whether $A \equiv B$. This is *subsumption*, the single point at which conversion is invoked from checking:

$$\frac{\Gamma \vdash e \Rightarrow B \qquad \Gamma \vdash A \equiv B}{\Gamma \vdash e \Leftarrow A}\ \textsc{Sub}$$

To see the modes in motion, consider checking

$$f : \mathrm{Nat} \to \mathrm{Bool}, a : \mathrm{Nat} \vdash f\ a \Leftarrow \mathrm{Bool}.$$

Application is not a checking form, so subsumption fires. Synthesis on $f\ a$ synthesises the head

$$f : \mathrm{Nat} \to \mathrm{Bool}, a : \mathrm{Nat} \vdash f \Rightarrow \mathrm{Nat} \to \mathrm{Bool},$$

then checks the argument

$$f : \mathrm{Nat} \to \mathrm{Bool}, a : \mathrm{Nat} \vdash a \Leftarrow \mathrm{Nat},$$

which descends into another synthesis and subsumption. The application as a whole synthesises

$$f : \mathrm{Nat} \to \mathrm{Bool}, a : \mathrm{Nat} \vdash f\ a \Rightarrow \mathrm{Bool},$$

and conversion checks $\mathrm{Bool} \equiv \mathrm{Bool}$ trivially.

For dependent function application $f\ a$ with $f \Rightarrow \Pi_{x:A} B$, the result type is $B[a/x]$. The substitution forces the conversion checker, which compares types modulo $(\beta, \delta, \zeta, \eta, \iota)$, to evaluate terms inside types. The next section gives the algorithm that does so.

2.3 Normalisation by evaluation

Conversion calls from Section 2.2 must reduce terms to compare them. A naive implementation would substitute syntactically on every β-step, duplicating work whenever a substituted variable later reappears, and shifting de Bruijn indices across each binder. *Normalisation by evaluation* (NbE) [Abel, 2013] avoids both costs: it interprets terms into a *semantic* domain of *values* in which application is a meta-language operation, and substitution is replaced by environment lookup. Two functions complete the round trip:

$$\mathsf{eval} : \mathsf{Term} \times \mathsf{Env} \to \mathsf{Value}$$
$$\mathsf{quote} : \mathsf{Value} \to \mathsf{Term}$$

Evaluation reduces a term to *weak head normal form* (WHNF) under an environment ρ binding each free variable to a value. *Head* means the outermost form is decided (canonical or stuck), and *weak* means evaluation does not enter the bodies of λ-binders, which stay packaged as closures, evaluated only when an argument arrives.

WHNF suffices for conversion: at matching canonical heads, the algorithm recurses into the subterms (forcing closures as needed); at neutrals, it compares head and spine. Quotation walks a value back to a term, descending into closures by applying them to a fresh variable.

A value is either *canonical* (a function lam, a type code pi' or u', or an inductive constructor applied to its arguments) or neutral: a stuck head h (a free variable, or a constant whose body has not been unfolded) paired with a *spine* σ, a list of arguments and projections accumulated since the last reduction of the head. The canonical value of a lambda carries a *closure* $\langle \rho, t \rangle$: the body t paired with the environment ρ of its captured variables, not yet evaluated.

Three rules give the character of evaluation:

$$\begin{aligned} \mathsf{eval}(x_i, \rho) &= \rho(i) \\ \mathsf{eval}(\lambda(x : A).t, \rho) &= \mathsf{lam}(x, \mathsf{eval}(A, \rho), \langle \rho, t \rangle) \\ \mathsf{eval}(t\ u, \rho) &= \mathsf{app}(\mathsf{eval}(t, \rho), \mathsf{eval}(u, \rho)) \end{aligned}$$

with the auxiliary app:

$$\begin{aligned} \mathsf{app}(\mathsf{lam}(x, A, \langle \rho, t \rangle), v) &= \mathsf{eval}(t, \rho \cdot v) \\ \mathsf{app}(\mathsf{neutral}(h, \sigma), v) &= \mathsf{neutral}(h, \sigma \cdot v) \end{aligned}$$

The closure case fires β inside the meta-language; the neutral case extends the spine without reducing. Quotation at a neutral reverses the spine back to a syntactic application; at a closure, it applies to a fresh variable and recurses under the binder.

A defined constant evaluates to a *glued* value, which carries the folded neutral (the constant applied to its universe arguments, empty spine) together with the constant's identifying information $\big(c, \overline{\ell}\big)$ used to fetch and evaluate the body via δ-reduction when needed:

$$\mathsf{eval}\big(c.\{\overline{\ell}\}, \rho\big) = \mathsf{glued}\big(\mathsf{neutral}\big(c.\{\overline{\ell}\}, \varepsilon\big), c, \overline{\ell}\big)$$

Conversion compares two glued values' folded heads and spines first; when those agree, the bodies are not fetched. When they disagree, whnf forces δ-reduction and conversion repeats on the unfolded values. We use glued evaluation for two reasons: the cheap-comparison-first discipline is due to Kovács's smalltt [Kovács, 2024, 2023] and is the practical performance argument; and each δ-reduction in whnf is the elaborator's act of depending on c's body, which the build framework records as a cross-declaration dependency edge (Section 3.2).

To see evaluation on a small term, consider $(\lambda x.f\ x)\,a$ at the empty environment, with f a constant whose body is unavailable:

$$\begin{aligned} \mathsf{eval}((\lambda x.f\ x)\,a, \varepsilon) &= \mathsf{app}(\mathsf{lam}(x, \_, \langle \varepsilon, f\ x \rangle), \mathsf{eval}(a, \varepsilon)) \\ &= \mathsf{eval}(f\ x, \varepsilon \cdot a) \\ &= \mathsf{app}(\mathsf{neutral}(f, \varepsilon), a) \\ &= \mathsf{neutral}(f, \varepsilon \cdot a) \end{aligned}$$

Quotation reads this back as $f\ a$. Section 3.4.5 describes our implementation.

2.4 Build systems

We adopt the framework of *Build systems à la carte* (BSALC) [Mokhov et al., 2020], which abstracts the build recipe as a polymorphic Task type. This section introduces the vocabulary; Section 2.5 covers existing instances; Section 3.2 presents the refinement we make in this thesis.

2.4.1 The Task abstraction

An elaborator fits the build-system shape: source files are inputs, and declarations are queries whose recipes are tasks. A build system computes the value of each *key* from the values of other keys, bottoming out at input keys whose values are read from the outside world. The recipe for non-input keys is a *task*:

$$\mathsf{Task}\, c\, k\, v = \forall f.\quad c\, f \Rightarrow (k \to f\, v) \to f\, v.$$

A task is parametric in a callback monad f. It receives a callback $\mathsf{fetch} : k \to f\, v$ that resolves any dependency in f, and returns a value in f. The constraint c ranges over `Applicative`, `Selective`, `Monad`, and others; each corresponds to a discipline of dependency tracking. With `Applicative`, the set of fetched keys is fixed up front. With `Monad`, fetches may be conditional on the values returned by earlier fetches: this is the discipline elaboration needs, because conversion checking discovers cross-declaration dependencies only as it unfolds constants and compares terms.

Polymorphism in f separates *what* a build computes from *how* a particular runtime schedules and caches it. The same Task can be executed pure (in `Id`, recomputing everything), or in a stateful monad that memoises, or under additional effect layers that trace or cancel.

2.4.2 Towards a verified build framework

The published Task, embedded into Lean 4 for a machine-checked agreement theorem against a batch reference, is missing four things:

- **Dependent result types.** Different queries return different value types: querying the `ast` of a file expects an `Ast` type, and querying the `type` of a constant expects a `Ty`. Whilst the original meaning of a *build system* is usually not concerned with more than associating filenames and their contents, it is not sufficient for our use case.
- **Well-foundedness.** The batch reference recurses through whatever dependencies a task fetches. For this recursion to define a total function in Lean, the framework must carry a well-founded relation on queries.
- **Structural parametricity.** BSALC's Task is polymorphic in f so that different build systems may instantiate it with different effect monads. Wadler's metatheorem [Wadler, 1989] says any polymorphic term's two instantiations agree on related handlers. Whilst Haskellians appeal to this informally, treating it as a property of the type, Lean's logic does not derive such metatheorems internally.
- **Effect orthogonality.** Tracing and cancellation, added as outer effect layers, must compose with the agreement proof without re-running it. The correctness statement must be polymorphic in those layers.

We refine BSALC's polymorphic Task into a verified build framework that supplies all four. Section 3.2 presents the framework; Section 3.3 exhibits three cached executors that inhabit it.

2.5 Existing approaches

2.5.1 Prefix elaboration

Production proof assistants invalidate at the file boundary. Lean 4 [de Moura and Ullrich, 2021], Agda [Norell and Chapman, 2009], and Rocq [The Coq Development Team, 2024] elaborate top to bottom, accumulating a global environment. The cache key is the program prefix up to position p; an edit before p invalidates the cache. Lean represents this as a snapshot tree per file [Ullrich, 2023]; Agda caches each successful declaration; Rocq re-checks from the first modified sentence. Across files the granularity is coarser: any change to an imported module invalidates the importer. The global environment is the only channel by which one declaration sees another, and the elaborator does not record which earlier declarations are relevant, so every declaration nominally depends on the entire prefix.

2.5.2 In adjacent settings

A *query* is a pure function from a key to a value, registered with a runtime that memoises its result. For each completed query the runtime records the value and the keys it fetched as dependencies; the dependency graph is built when the query runs, not declared upfront. Queries come in two kinds. *Input queries* have no recipe; their values are set externally. *Derived queries* are pure functions of other queries, fetched on demand. When an input changes, the runtime marks transitively dependent derived queries potentially stale; when a stale query is recomputed, its new value is compared against the cached one, and if they agree the cascade stops. This is *early cutoff*. Together these give recomputation proportional to the affected fragment. The vocabulary is rooted in self-adjusting computation [Acar et al., 2002] and Adapton [Hammer et al., 2014], with *demand*, *dirtying*, and *cleaning* taken from Make's lineage [Feldman, 1979].

Salsa [Matsakis and contributors, 2018] is a Rust framework for on-demand query evaluation along these lines. Its scheduling discipline is request-driven, with a dirty bit set on fetch and no verifying trace over the cache. The programmer declares queries and their dependencies in a database trait, and the framework generates the storage, memoisation, and invalidation logic. rust-analyzer [Kladov and contributors, 2020], the language server for Rust, uses Salsa to implement its analysis pipeline as a query graph. Edits update input queries; IDE requests (hover, completion, find references) are derived queries that read from the cache.

In Rust, the type signature is the compilation boundary: a body edit cannot change the type-checking of any other function, because the elaborator only needs the callee's signature to check the caller. Salsa keys its caches by the signature; a body edit invalidates the cache for the edited function only. In dependent type theory the boundary is different. The conversion rule allows declaration N's elaboration to δ-unfold the body of an earlier declaration M; if M's body changes, N's elaboration result may change even though M's signature does not. The dependency from N to M is on whichever δ-reductions N's conversion checks perform, discovered as elaboration runs.

Sixty [Fredriksson, 2019] is an experimental Haskell elaborator for a dependently typed language. It records the constants its conversion checker unfolds and uses that record for invalidation. The cache is in-memory within a single language-server process.

Incremental type checking has also been studied for simpler systems [Meertens, 1983, Pacak et al., 2020, Porter et al., 2025]; the gap addressed in this thesis is doing so for a dependent type theory.

2.6 Requirements analysis

The elaborator and its build substrate must deliver five things. The proposal asked for two: correctness and incremental responsiveness. The other three follow from wanting the framework's agreement with batch elaboration to be a machine-checked theorem rather than an external appeal (Section 2.4.2).

2.6.1 Desiderata

Correctness The elaborated form of every declaration is the same whether the elaborator runs from cold or from a cached build. (Section 4.1)

Incremental responsiveness Re-elaboration time scales with the affected fragment, not with the size of the corpus. (Section 4.3.)

Verifiability The build substrate admits a machine-checked agreement theorem relating cached and batch elaboration. (Section 4.1.1.)

Executor polymorphism Modern incremental frameworks allow execution strategies to be swapped at deployment time; the elaborator is therefore presented as a single value, independent of the cache strategy that executes it. Switching strategies requires no source change and no rerun of the agreement proof. (Section 3.3.)

Effect orthogonality Modern tooling expects effect layers around the verified core; the framework therefore admits them without revisiting the agreement proof. (Section 3.3.4.)

2.7 Software engineering practices

2.7.1 Development methodology

The project decomposes into modular components: the elaborator, the build system, verification, corpora, and effect layers. Extensions to these were largely orthogonal, which made the spiral model of software development [Boehm, 1988] a natural fit. The elaborator and the polymorphic-`Task` build system came first, each delivering a usable artefact (an elaborator passing feature programs; a cached executor running it) before the next iteration began. Verification of the agreement theorem, evaluation corpora, and the effect-layer extensions followed in subsequent iterations. Within an iteration, priority went to items on the path to a success criterion of the proposal. Risk was assessed by whether a feature had a published implementation or specification (low risk: bidirectional checking, normalisation by evaluation, BSALC's polymorphic `Task`) or required novel work (higher risk: structural parametricity certificates). Debug instrumentation for the build framework, specifically, query tracing, was prioritised early for ease of development.

2.7.2 Version control and tooling

Source code and dissertation are tracked in Git, with the repository hosted on GitHub. The Lean toolchain `Lake` manages the build and dependencies; `mathlib` [The mathlib Community, 2020] is the only explicitly required package, with the Lean community `Cli` library and others pulled in transitively. The language server is built against modules from `Lean.Data.Lsp`, and the command-line interface against `Cli`.

2.7.3 Code style

Despite Lean having no canonical formatter, we strive for consistent style across the codebase. Where the type system can enforce an implementation invariant, we let it: terms and types are intrinsically well-scoped, indexed by the number of binders they preserve, so out-of-bounds access is a type error. Tests are defined inline using Lean's `#eval`, `#guard_msgs`, and `#guard` directives and custom meta-programming (Section 4.1), elaborating through the same code path users invoke rather than against a parallel harness.

2.7.4 Language choice

The proposal named Salsa [Matsakis and contributors, 2018] in Rust as the incremental substrate. At the start of implementation the decision was revisited; Lean 4 and OCaml were considered alongside. Lean 4 was chosen for three reasons. The query layer is dependently typed (`Val : Key → Type`): Lean expresses this directly, Rust has neither dependent types nor GADTs (generalised algebraic data types, whose constructors specialise the type parameter), and OCaml has GADTs but is not dependently typed and so cannot express proofs. One project aim is a machine-checked correctness proof relating cached and batch elaboration; in Lean the implementation and its proof live in the same project. In addition, Lean's meta-programming features and introspectability of its own compiler made it easy to prototype ideas, and to write tests.

Salsa was therefore replaced by a polymorphic-`Task` abstraction [Mokhov et al., 2020] written in Lean (Section 2.4.2). A Salsa-style inhabitant is retained as a stretch deliverable, running against the same `Tasks` value for comparison.

2.7.5 Licensing and open source

Our code is licensed under the open source Apache 2.0 license, which is aligned with the Lean ecosystem. As discussed in Section 4.2, we patch the Lean 2 compiler, which is itself Apache 2.0, to add common-subexpression elimination in order to translate the Lean 2 HoTT library [The Lean 2 contributors, 2017] efficiently. Apache 2.0 permits this modification and redistribution; hence, we retain the copyright headers, enumerate the modified files, and include the relevant attribution in a NOTICE file.

2.8 Starting point

Prior to starting, I had implemented type checkers, but had not built a dependently typed elaborator or a build system. Although I had experience using Lean 4 as a proof assistant, but had not used it as a general-purpose language for a project of this scale, nor used its metaprogramming features.

No implementation code was written before the project began, and no existing codebase was used as a basis. The proposal planned a Rust implementation on top of Salsa; no code from that stack was incorporated. smalltt [Kovács, 2023] was consulted as a reference for the design of *glued conversion checking* (Section 3.4.5). Our language server borrows code from Lean 4's.

3 Implementation

Section 3.1 lays out the elaboration pipeline that the rest of the chapter populates. Section 3.2 formalises the `Build` interface in Lean 4, with `Task`'s parametricity certificate as the device by which an executor's agreement with the reference `compute` semantics is proved inside the language. Section 3.3 exhibits four verified cached executors — `Busy`, `LessBusy`, `Shake`, and `ShakeStandardRdeps` — and two effect-layer extensions derived from `Shake` without revisiting the proof, alongside three unverified native implementations.

Section 3.4 presents the elaborator as a single `Tasks` value: parsing to a green tree, query-based decomposition, bidirectional checking, normalisation by evaluation, conversion's three-mode algorithm, inductive declarations, and the per-declaration incremental wiring. Section 3.5 wraps the executor for editor interaction. Section 3.6 indexes the source.

3.1 Design goal

A machine-checked agreement theorem between cached and batch elaboration requires the four properties of Section 2.4.2. This chapter constructs a framework that satisfies them, instantiates it with three caching strategies, and wires the elaborator into it.

The construction proceeds in four steps. Section 3.2 defines the `Build` interface and the `compute` reference semantics that every inhabitant must agree with, with the parametricity certificate carried as a field of `Task`. Section 3.3 exhibits three inhabitants: `Busy` runs `compute` directly, `LessBusy` adds within-build memoisation, and `Shake` adds cross-build persistence with verifying traces. Two effect-layer extensions, `ShakeTrace` and `ShakeCancel`, are derived from `Shake` without revisiting the agreement proof. Section 3.4 presents the elaborator as a single `Tasks` value, independent of which inhabitant executes it, and records the constants the conversion checker unfolds so that invalidation is keyed on actual dependencies rather than file boundaries. Section 3.5 wraps the executor for editor interaction.

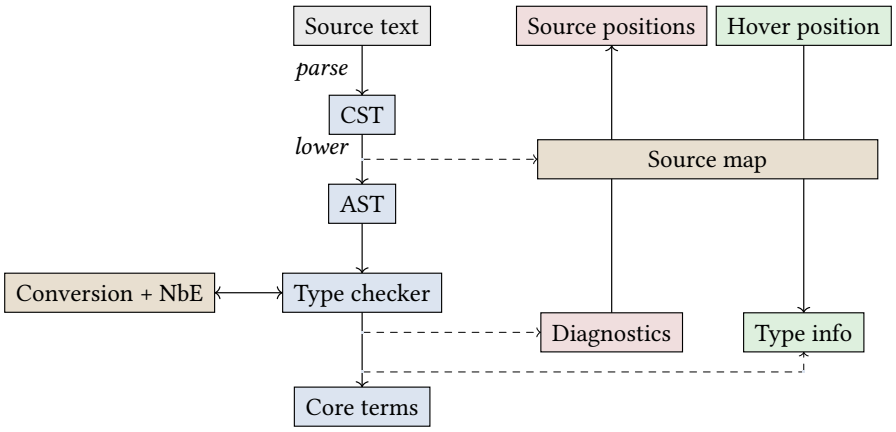

Figure 1: Elaboration pipeline. Source text is parsed to a CST, then lowered to an AST and a source map between CST and AST paths. The type checker produces core terms and emits diagnostics keyed by AST paths. Diagnostics are mapped through the source map to recover source positions.

3.2 The build framework

This section presents the Lean 4 formalisation that adds the four properties of Section 2.4.2. Dependent result types and structural parametricity are carried in the framework's representation; well-foundedness lives in a separate reference semantics; effect orthogonality is achieved by stacking effect layers around a proof-relevant payload. Cross-build persistence follows the verifying-trace design of Shake [Mitchell, 2012]. Section 3.3 presents the cached executors that inhabit the framework.

3.2.1 Setup

Parsing returns a syntax tree, elaboration returns a typed term, diagnostics return an array of errors. Each query has its own result type. We separate the key space into *inputs* `I` (set externally) and *queries* `Q` (computed), each with a dependent value type:

$$V : I \to \text{Type}, \quad R : Q \to \text{Type}.$$

Termination needs a well-founded relation on queries; a task at query q_0 may only fetch queries q with `rel q q₀`. The fields bundle into a configuration:

```
structure BuildConfig : Type 1 where
  I : Type;  V : I → Type
  Q : Type;  R : Q → Type
  rel : Q → Q → Prop
  wf  : WellFounded rel
```

An input store is a value `J` with a typeclass providing `get` and `set` operations satisfying `get_set_self` and `get_set_other` (set updates exactly one input). The simplest instance is `Function.update` on `∀ i, 𝒞.V i`; the file-system instance reads and writes paths.

3.2.2 Task

A task specifies how to produce one query's result. It consumes inputs and dependencies (any query `q` strictly preceding q_0 under `𝒞.rel`) and returns a value of the target type. A task is described once and runs under any monad: pure recomputation under `Id`, a stateful build under `StateM Cache`, a cancellable build under `CancelM Cache`.[9]. Caching, scheduling, and execution order are the responsibility of inhabitants of `Build`, rather than the task.

A task is polymorphic over its effect monad as in BSALC's published Task, so that different build systems may instantiate it with different `f`. Wadler's metatheorem [Wadler, 1989] says any polymorphic term's two instantiations agree on related handlers; Haskellians appeal to this informally as a property of the type. Lean's logic does not derive such metatheorems internally, because a polymorphic Lean term may inspect its type argument through typeclass dispatch. To use the fact inside a Lean proof, each task carries a relational certificate at construction. The certificate is quantified over a *monad action* [Voigtländer, 2009], the device Voigtländer introduced in 2009 for binary parametricity at a type constructor of higher kind: a relation lifter between two monads preserving `pure` and `>>=`. The framework exposes the relational obligation as data, with each task carrying a constructive proof against this interface:

[9] This unfolds to `ReaderT (BaseIO Bool) (ExceptT Cancelled (StateT Cache BaseIO))`, that is, a monad with the capability to read a boolean flag from another thread, possibly returning failure if cancelled, but always returning an updated cache.

```
structure MonadAction (κ₁ κ₂ : Type → Type) [Monad κ₁] [Monad κ₂] where
  rel       : (α → β → Prop) → (κ₁ α → κ₂ β → Prop)
  rel_pure  : R a b → rel R (pure a) (pure b)
  rel_bind  : rel R ma mb → (∀ a b, R a b → rel S (ka a) (kb b))
            → rel S (ma >>= ka) (mb >>= kb)
```

`rel` sends a value relation `R : α → β → Prop` to a computation relation `rel R : κ₁ α → κ₂ β → Prop`. The two laws require that pure values from related inputs are related, and that binds preserve relatedness over pointwise-related continuations.

A `Task 𝒞 q₀ α` then bundles a *polymorphic computation* `fn` with the certificate `param`:

```
structure Task (𝒞 : BuildConfig) (q₀ : 𝒞.Q) (α : Type) : Type 1 where
  fn    : ∀ (f : Type → Type) [Monad f],
          (∀ i, f (𝒞.V i)) → (∀ q, 𝒞.rel q q₀ → f (𝒞.R q)) → f α
  param {κ₁ κ₂ : Type → Type} [Monad κ₁] [Monad κ₂]
    (A : MonadAction κ₁ κ₂)
    {ι₁ : ∀ i, κ₁ (𝒞.V i)} {ι₂ : ∀ i, κ₂ (𝒞.V i)}
    (f₁ : ∀ q, 𝒞.rel q q₀ → κ₁ (𝒞.R q))
    (f₂ : ∀ q, 𝒞.rel q q₀ → κ₂ (𝒞.R q)) :
    (∀ i, A.rel Eq (ι₁ i) (ι₂ i)) →
    (∀ q hq, A.rel Eq (f₁ q hq) (f₂ q hq)) →
    A.rel Eq (fn κ₁ ι₁ f₁) (fn κ₂ ι₂ f₂)
```

`fn` takes any monad `f`, a way to read each input as an `f`-computation, and a way to fetch each dependency as an `f`-computation. The output has type `f α`. Monad polymorphism abstracts the runtime: the body of `fn` can only sequence reads and fetches through `pure` and `bind`.

`param` reads: for any monad action `A` between `κ₁` and `κ₂`, if the input and fetch handlers are pointwise `A.rel Eq`-related, the two instantiations of `fn` produce `A.rel Eq`-related results. The certificate is discharged constructively at each primitive (`Task.pure`, `Task.bind`, `Task.input`, `Task.fetch`); compositions inherit theirs via `do`-notation, so a task written in monadic style obtains one without explicit proof obligation.

The dependency constraint `𝒞.rel q q₀` is encoded in the type of the fetch handler: a task cannot ask for itself or for any cyclic dependency. The recursive definition of `compute` (below) terminates by induction on `𝒞.wf`. A configuration with cyclic dependencies has no terminating evaluation; requiring `𝒞.wf` rules out exactly those configurations.

3.2.3 Specification

A family of tasks indexed by target query is `Tasks 𝒞 := ∀ q, Task 𝒞 q (𝒞.R q)`. The reference semantics `compute` runs each task in `Id`, recursively recomputing every dependency with no memoisation:

```
def compute (tasks : Tasks 𝒞) (ι : ∀ i, 𝒞.V i) (q : 𝒞.Q) : 𝒞.R q :=
  (tasks q).fn Id ι (fun q' _ ⇒ compute tasks ι q')
termination_by 𝒞.wf.wrap q
```

A `Value` pairs a result with a proof it equals `compute`:

```
structure Value (tasks : Tasks 𝒞) (ι : ∀ i, 𝒞.V i) (q : 𝒞.Q) where
  val  : 𝒞.R q
  spec : val = compute tasks ι q
```

`Build` is the interface every cached executor implements. The state σ advances as the build runs; the outer monad `n` carries execution effects; the inner `m` is a monad for tracing or cancellation. Correctness is independent of `n` and `m`, holding via the `spec` field of `Value`.

```
structure Build (𝒞 : BuildConfig) (J : Type) [Input 𝒞 J] (tasks : Tasks 𝒞)
    (n m : Type → Type) : Type 1 where
  σ      : Type
  init   : J → σ
  inputs : σ → ∀ i, 𝒞.V i
  set    : ∀ i, 𝒞.V i → StateM σ Unit
  build  : ∀ q store, n (m (Value tasks (inputs store) q) × σ)
```

Any inhabitant of `Build` produces the same result as `compute`. Section 3.3 exhibits three: `Busy` (no caching), `LessBusy` (intra-build), and `Shake` (cross-build verifying traces).

3.2.4 Theorems for a fee

`LessBusy` and `Shake` run tasks in `StateM Cache`; correctness is stated against `compute`, which runs them in `Id`. The fee paid at task construction buys a framework-level lemma `Tasks.freeTheorem`: for any monad action with `Id` as the second monad, given pointwise-related inputs and fetches, the task result in the first monad is related to `compute` at the top-level query.

```
theorem Tasks.freeTheorem (tasks : Tasks 𝒞) (q₀ : 𝒞.Q)
    (F : MonadAction κ Id) (ι₀ : ∀ i, 𝒞.V i) (ι₁ : ∀ i, κ (𝒞.V i))
    (fetch₁ : ∀ q, 𝒞.rel q q₀ → κ (𝒞.R q))
    (hι     : ∀ i,    F.rel Eq (ι₁ i)       (ι₀ i))
    (hfetch : ∀ q hq, F.rel Eq (fetch₁ q hq) (compute tasks ι₀ q)) :
    F.rel Eq ((tasks q₀).fn κ ι₁ fetch₁) (compute tasks ι₀ q₀)
```

The proof has two steps. Apply the task's `param` with `F`, the supplied fetches in κ, and `fun q _ ⇒ compute tasks ι₀ q` as the second fetch family; the hypotheses `hι` and `hfetch` discharge the relatedness side-conditions. This yields

$$F.\mathrm{rel}\ \mathrm{Eq}\ ((\mathrm{tasks}\ q_0).\mathrm{fn}\ \kappa\ \iota_1\ \mathrm{fetch}_1)\ ((\mathrm{tasks}\ q_0).\mathrm{fn}\ \mathrm{Id}\ \iota_0\ (\lambda q.\ \mathrm{compute}\ \mathrm{tasks}\ \iota_0\ q)).$$

Unfolding `compute` once on the right rewrites this to `compute tasks ι₀ q₀`. The certificate carried by each task is what turns this from an external appeal to parametricity into a theorem in Lean.

`LessBusy` and `Shake` supply the specific monad action, the *cache action*, relating stateful computations to pure values by "return the same value from every starting state":

```
def cacheAction : MonadAction (StateM (VCache tasks ι₀)) Id where
  rel P m b := ∀ s, P (m.run' s) b
  rel_pure hab _ := hab
  rel_bind hma hk s := hk _ _ (hma s) _
```

With this action, `Tasks.freeTheorem`'s conclusion specialises to: if every cache entry returned by `fetch` equals `compute` at its query, the cached task result equals `compute` at q_0. The full Shake proof, including the cross-input invariant that lets memos survive across builds, is in A.

3.3 Build inhabitants

Four inhabitants of `Build` are verified against `compute`: `Busy` runs `compute` directly, `LessBusy` adds within-build memoisation, `Shake` adds persistent cross-build verifying traces, and `ShakeStandardRdeps` augments the persistent cache with reverse-dependency tracking so that an unchanged input short-circuits the trace walk entirely. Two effect-layer extensions, `ShakeTrace` and `ShakeCancel`, inherit the agreement theorem without revisiting the proof.

3.3.1 Busy

`Busy` calls `compute` directly. The build is from scratch each time and there is no cache. The agreement proof is by definition.

```
def Busy (tasks : Tasks 𝔎) : Build 𝔎 J tasks Id Id where
  σ := J;  init := id;  inputs := Input.get
  set i v := modify (Input.set · i v)
  build q store := (⟨compute tasks (Input.get store) q, rfl⟩, store)
```

3.3.2 LessBusy

`LessBusy` caches results within a single build, so each query is computed at most once per build. Each cache entry is a `Value`, which carries a proof that it equals `compute`. The `fetch` function checks the cache first; on a miss, it runs the task, caches the result, and returns it:

```
def fetch (q₀ : 𝔎.Q) :
    StateM (VCache tasks ι₀) (Value tasks ι₀ q₀) := do
  if let some v := (← get).get? q₀ then return v
  let v ← run tasks ι₀ q₀ (fun q _ ⇒ fetch q)
  modify (·.insert q₀ v)
  return v
termination_by 𝔎.wf.wrap q₀
```

Termination follows from the well-founded relation on queries. The `run` function executes the task and produces a proven `Value` via the free theorem.

The proof supplies the cache action of Section 3.2.4. Each cached fetch returns the same value as `compute` (the cache entries carry their own proofs), so `Tasks.freeTheorem` gives the overall result.

The top-level definition ties `fetch` into the `Build` type:

```
def LessBusy (tasks : Tasks 𝔎) : Build 𝔎 J tasks Id Id where
  ...   -- σ, init, inputs, set as in Busy
  build q store :=
    let ι₀ := Input.get store
    let (v, _) := LessBusy.fetch tasks ι₀ q (DHashMap.emptyWithCapacity 1024)
    (v, store)
```

An empty cache is passed as the initial state. Each `fetch` call populates it, and the returned `v` carries a proof that it equals `compute`.

3.3.3 Shake

`LessBusy` starts fresh on each build. `Shake` persists the cache across builds, using fingerprints to determine which entries are still valid. Dependencies are tracked by dedicated records:

```
structure InputDepHash (I H : Type) where
  key : I;  hash : H
structure QueryDepHash (𝔊 : BuildConfig) (q₀ : 𝔊.Q) (H : Type) where
  q : 𝔊.Q;  rel : 𝔊.rel q q₀;  hash : H
```

Each cached `Memo` carries a universally quantified invariant over these records:

```
structure Memo (q : 𝔊.Q) where
  value     : 𝔊.R q
  inputDeps : Array (InputDepHash 𝔊.I H)
  queryDeps : Array (QueryDepHash 𝔊 q H)
  invariant :
    ∀ (ι : ∀ i, 𝔊.V i),
      (∀ p ∈ inputDeps, hI p.key (ι p.key) = p.hash) →
      (∀ p ∈ queryDeps, hR p.q (compute tasks ι p.q) = p.hash) →
      value = compute tasks ι q
```

The invariant says: for *any* input function `ι`, if every recorded input fingerprint matches `ι` and every recorded dependency fingerprint matches `compute` under `ι`, then `value` equals `compute tasks ι q`. The universal quantification over `ι` is what lets the same memo be valid across builds with different inputs.

On recompute the task evaluates as a free monad tree [Swierstra, 2008], and the invariant proof uses `FM.evalTree_cross`: if two sets of inputs and dependencies agree at every position recorded in the trace (checked via injective embeddings $h_I : \forall i.\mathfrak{C}.Vi \hookrightarrow H$ and $h_R : \forall q.\mathfrak{C}.Rq \hookrightarrow H$), the free monad tree evaluates to the same result. The injectivity of the hash functions is what makes fingerprint comparison sound: `hash a = hash b` implies `a = b`. The verified Shake takes the embeddings as parameters; the elaborator instantiates them with the standard `Hashable`.

The verified Lean `Shake` lives in `Incremental/Shake/` and uses `runST` with mutable references for the memo table and in-progress cache.

3.3.4 Effect layers

The `Build` type carries two type constructors `n` and `m` that wrap each query's result:

```
build : ∀ q store, n (m (Value tasks (inputs store) q) × σ)
```

`n` is an outer effect threaded around the entire build step; `m` is an inner effect carried alongside the proven `Value`. `Busy`, `LessBusy`, and the basic `Shake` of Section 3.3.3 all instantiate both to `Id`, the trivial effectless monad. Replacing one or the other with a richer monad gives a verified build system with new observable behaviour, without revisiting the correctness proof: the `Value` payload is unchanged, so its `spec` field still witnesses agreement with `compute`.

Tracing. `ShakeTrace` instantiates `n` with `TraceT 𝔊.Q m`, a state transformer over a forest of dependency nodes recording the query, its wall-clock duration, and its child queries:

```
inductive DepNode (Q : Type) where
  | mk (q : Q) (durationNs : Nat) (children : Array (DepNode Q))

abbrev Forest (Q : Type) := Array (DepNode Q)
abbrev TraceT (Q : Type) (m : Type → Type) := StateT (Forest Q) m
```

Each invocation of the `Shake` fetch passes through `bracket`, which reads a monotonic clock before and after the inner computation, then pushes a fresh `DepNode` onto the surrounding trace:

```
def bracket (q : Q) (body : TraceT Q m α) : TraceT Q m α := fun outer ⇒ do
  let t0 ← IO.monoNanosNow
  let (a, inner) ← body #[]
  let t1 ← IO.monoNanosNow
  pure (a, outer.push (.mk q (t1 - t0) inner))
```

`ShakeTrace` is the standard `Shake` with `bracket` supplied to a generic `Build` parametrised by an `m` that supports the outer state and `IO`. The result of a build is a `Forest 𝒞.Q` whose shape mirrors the dynamic dependency tree.

Cancellation. `ShakeCancel` instantiates `m` with `Except Cancelled` and constrains the surrounding monad with a `MonadCancel` class:

```
class MonadCancel (m : Type u → Type v) where
  CanCancel {α} : m α → Prop
  checkpoint : m PUnit
class LawfulMonadCancel (m) [Monad m] [MonadAttach m] [MonadCancel m] : Prop
where
  canCancel_bind_imp : CanCancel (ma >>= k) →
    CanCancel ma ∨ ∃ a, CanReturn ma a ∧ CanCancel (k a)
  not_canCancel_pure : ¬ CanCancel (pure a : m α)
```

The `Id` monad has `CanCancel _ := False`, so the cancellation interface is trivially satisfied when no cancellation is warranted. For a cancellable monad (such as one that reads an `IO.Ref Bool`), `checkpoint` reads the cancellation flag and throws `Cancelled` if it is set. The `Shake` fetch loop is rewritten to call `MonadCancel.checkpoint` before each cache lookup:

```
def fetchCancel (persist) (ι₀) (q₀) := do
  monadLift (MonadCancel.checkpoint : m PUnit)
  let (vcache, cache) ← get
  if let some ⟨(v, _), _⟩ := vcache.get? q₀ then return v
  …
```

Memos completed before the checkpoint that raised are persisted via the supplied `persist` callback, so the next build sees a populated store and re-fetches only the queries that were in flight. The top-level `ShakeCancel` packages this fetch into a `Build 𝒞 J tasks BaseIO (Except Cancelled)`; an inhabitant by construction, so the correctness theorem of Section 3.2.4 applies unchanged.

3.3.5 Reverse dependencies

The basic `Shake` of Section 3.3.3 verifies every cached entry on every build, since the only persistent state is the trace of input and query fingerprints. A no-op edit still walks the whole cache (Section 4.3). `ShakeStandardRdeps` adds a fourth verified inhabitant that records reverse-dependency edges as the build runs, and uses them at `set` time to mark the transitive closure of *possibly-dirty* queries. A subsequent `build` skips both the input-fingerprint check and the recursive query-fingerprint walk for any query not in the dirty set.

The procedure is three steps. On recompute of a query q, for each input i it read append (i, q) to `inputRdeps`, and for each query d it fetched append (d, q) to `queryRdeps`. On `set i v` whose hash

differs from the cached input, seed the dirty set with every q such that $(i, q) \in$ `inputRdeps`, then walk forward under `queryRdeps` to its transitive closure: every cached query that could have read i, directly or through a chain of fetches, becomes *possibly-dirty*. On `build q`, if q is not in the dirty set and the cache has an entry for q, return it with no fingerprint check; otherwise recompute, append the new rdeps, and drop q from the dirty set. Verification cost becomes O(invalidation set) instead of O(cache).

The persistent state extends Shake's cache with three arrays:

```
structure Persist where
  cache      : DHashMap 𝔊.Q (Memo hI hR tasks)
  queryRdeps : Array (𝔊.Q × 𝔊.Q)
  inputRdeps : Array (𝔊.I × 𝔊.Q)
  dirty      : Array 𝔊.Q
```

Each entry (d, q) in `queryRdeps` records that query q fetched d during its last recompute; each (i, q) in `inputRdeps` records that q read input i. The `dirty` array is the working set of queries whose cached memo cannot be trusted.

Correctness is the conjunction of four invariants on `Persist`:

```
structure WellFormed (ι₀) (p : Persist) : Prop where
  sound : ∀ q mm, p.cache[q]? = some mm → q ∉ p.dirty →
    (∀ d ∈ mm.inputDeps, hI d.key (ι₀ d.key) = d.hash) ∧
    (∀ d ∈ mm.queryDeps, hR d.q (compute tasks ι₀ d.q) = d.hash)
  complete : ∀ q mm, p.cache[q]? = some mm →
    (∀ d ∈ mm.inputDeps, (d.key, q) ∈ p.inputRdeps) ∧
    (∀ d ∈ mm.queryDeps, (d.q, q) ∈ p.queryRdeps)
  closed : ∀ q mm, p.cache[q]? = some mm → ∀ d ∈ mm.queryDeps, d.q ∈ p.cache
  downClosed : ∀ q mm, p.cache[q]? = some mm → q ∉ p.dirty →
    ∀ d ∈ mm.queryDeps, d.q ∉ p.dirty
```

`Sound` says every clean cached memo's stored fingerprints still match the current inputs and dependencies, so its universally quantified `invariant` field discharges agreement with `compute` immediately. `Complete` says each cached dependency edge is registered in the rdeps arrays, which is what lets `set` find every cached entry affected by an input change. `Closed` says the cache is transitively populated below every cached entry. `DownClosed` says cleanliness is downward-closed under the dependency relation, so a clean memo can call `Sound.value` on its dependencies without revisiting them.

The `Build`'s state type is a subtype carrying `WellFormed` as an irrelevant proof:

```
σ := { sp : J × Persist // WellFormed (Input.get sp.1) sp.2 }
```

`init` returns the empty `Persist`, where every invariant is vacuous via `DHashMap.get?_emptyWithCapacity`. `set i v` first checks whether the new input hashes the same as the old; by injectivity of h_I the input is then unchanged, and the existing `Persist` carries through with `WellFormed` transported by `Input.get j' = Input.get j`. Otherwise it appends the transitive rdeps closure of i to `dirty`:

```
def invalidate (p : Persist) (i : 𝔊.I) : Persist :=
  { p with dirty := p.dirty ++ closure p.queryRdeps (seedOf p.inputRdeps i) }
```

`closure` performs a bounded breadth-first walk over `queryRdeps`, with the bound `seed.size + rdeps.size + 1` justified by the nodup invariant on the visited set.

`invalidate_preserves_downClosed` then establishes that every queryDep of any cached entry that remains clean after invalidation was already clean before it, since the closure is closed under the rdeps edge.

`build q` calls `populate q`, defined by well-founded recursion on `𝒞.rel`. The fast path applies when q is not in `p.dirty` and the cache hits: `Sound.value` strips the memo's `invariant` field directly, returning the cached value with its agreement certificate. Otherwise `runWalk` traces the task tree as in Section 3.3.3, building a fresh `Memo` from the trace, and the four invariants are re-established on the inserted `Persist`:

- `Sound` and `Complete` extend across the insert via `recordRdeps_preserves_sound` and `recordRdeps_preserves_complete`, both routed through a single `cache_insert_cases` helper that case-splits on whether the lookup key equals the freshly inserted one.
- `Closed` and `DownClosed` lift from the runtime witnesses `hDepsInCache` and `hDepsNotDirty` carried by `runWalk`'s result: every query fetched during the recompute is now cached and clean.

Preservation of `Sound` across `invalidate` runs by well-founded recursion on `𝒞.rel`: at each cached entry still clean after invalidation, the inductive hypothesis at every queryDep reverifies its hash under the updated input (the dep is clean by `DownClosed` + `closure_step_closed`), and `mm.invariant` recovers agreement. The closure itself terminates by a lexicographic measure with an explicit fuel bound against the universe of queries reachable through `queryRdeps`. The full walkthrough is in Section B.

Section 4.3 reports `ShakeStandardRdeps` alongside the basic `Shake` and the unverified `shake-rdeps`.

3.3.6 Native implementations

Alongside the verified inhabitants, the repository ships three operational drop-in replacements: `Salsa`, a Lean rebuild-on-demand variant with reverse-dependency tracking; and `SalsaC` and `ShakeC`, C ports of `Salsa` and `Shake` compiled to native code and called through the Lean runtime ABI. All three implement the same `Build` interface as the verified inhabitants, so the elaborator and language server consume them interchangeably:

```
@[extern "lean_shake_build"]
opaque shakeCBuild [Input 𝒞 J] [BEq 𝒞.I] [Hashable 𝒞.I] [∀ i, Hashable (𝒞.V
i)]
    [BEq 𝒞.Q] [Hashable 𝒞.Q] [∀ q, Hashable (𝒞.R q)] :
    Tasks 𝒞 → ∀ q, ShakeRT.Store 𝒞 J → 𝒞.R q × ShakeRT.Store 𝒞 J

def ShakeC (tasks : Tasks 𝒞) : Build 𝒞 J tasks Id Id where
  ...   -- σ, init, inputs, set as in Shake
  build q store := let (r, s) := shakeCBuild tasks q store; (⟨r, sorry⟩, s)
```

`Salsa`, `SalsaC`, and `ShakeC` are `Build` inhabitants whose `build` field carries the agreement certificate as a `sorry`: the axiom that the underlying implementation computes the same value as `compute` on the same `Tasks`. The C ports are 459 and 404 lines respectively; they match Lean's runtime ABI (constructor field order and scalar layout for `Memo` and `Store`, closure arity for the `fetch` and `input` callbacks, reference counting protocol for allocated objects) and skip Lean's closure allocation and monadic bind dispatch on each query. They are selected when raw throughput is preferred over the structural proof carried by `Shake`. Section 4.3 reports them alongside `Shake`.

3.4 The elaborator

The elaborator is a single `Tasks` value (Section 3.2) whose queries decompose the work from source text down to elaborated kernel constants. Independence from the executor is structural: the same `Tasks` runs under `Busy`, `LessBusy`, `Shake`, and the effect-layer variants. This section walks through each query in the chain, ending with the unfolding-recording mechanism that lets `Shake`'s fingerprints invalidate exactly the declarations whose elaboration depended on the edited symbol.

3.4.1 Parsing

The parser is a hand-rolled Pratt parser [Pratt, 1973], implemented as combinators over `ParserFn := State → State`, where `State` carries the input string, current byte position, a stack of partial `Cst` nodes, and an accumulating error log. Source text is first parsed to a concrete syntax tree (`Cst`) retaining trivia and exact token widths, then lowered to an abstract syntax tree (`Ast`) that strips trivia and expands surface syntax.

3.4.1.1 Green trees

The CST follows the *green tree* pattern [Kladov and contributors, 2020, Microsoft, 2015]. Nodes and tokens are tagged by `SyntaxNodeKind`; positions are not stored:

```
inductive Cst : Type
  | token (kind : SyntaxNodeKind) (val : String)
  | node (kind : SyntaxNodeKind) (children : Array Cst)
with
  @[computed_field]
  width : Cst → Nat
    | .token _ val ⇒ val.length
    | .node _ children ⇒ children.map width ▷.sum
```

Nodes store only their kind and children, rather than absolute positions. Positions are recovered on demand by summing widths from the root. Because nodes lack positions, two identical subtrees are structurally equal regardless of where they appear in the file.

Separately, whitespace and comments are parsed as *trivia* tokens, stored as siblings of content tokens. Lowering discards trivia when producing the AST, so whitespace edits that only affect trivia tokens do not change the AST hash.

3.4.1.1.1 Example

Consider the file containing:

```
def a : Nat := Nat.zero
def b : Nat := a
```

The CST is a tree rooted at a `file` node, with trivia (whitespace, comments) and declaration subtrees as children. Each declaration subtree contains its tokens. Figure 2 shows the structure, with token children of each declaration abbreviated.

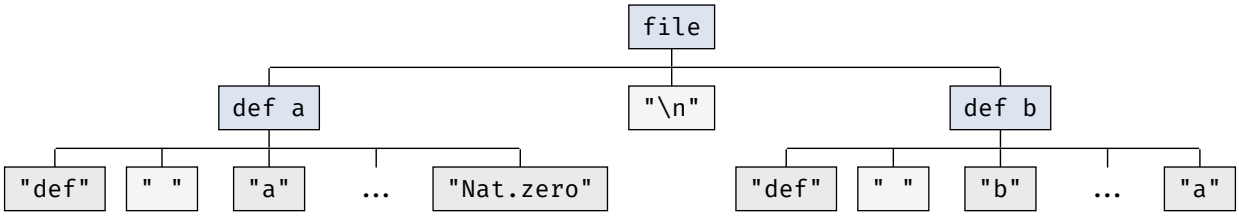

FIGURE 2: CST for the two-definition file. Whitespace is stored in separate trivia tokens.

3.4.1.1.2 TRIVIA AND POSITION INDEPENDENCE

Take the two-declaration file from Figure 2 and consider two edits. Adding extra spaces inside `def a` changes only the surrounding trivia token; the content tokens (`"def"`, `"a"`, `":="`, `"Nat.zero"`) are byte-for-byte identical, so the `def a` subtree hashes the same after lowering discards the trivia. Inserting a new declaration `c` between `a` and `b` leaves `b`'s subtree alone: a definition's subtree is determined by its tokens and their structure, and green tree nodes carry no absolute offsets.

The elaboration pipeline uses *paths*, lists of child indices relative to a subtree root, rather than absolute positions. A path `[2, 1]` means "child 2 of the subtree, then child 1 of that". Diagnostics and hovers are keyed by path relative to the declaration's subtree. After the `c` insertion, the `declarationIndex` query (which maps names to indices) recomputes because `b` sits at a different index, but `b`'s subtree hashes the same, so `elabDecl b` is reused from the cache. Only `c` is elaborated.

If nodes stored absolute byte offsets, every node after the insertion point would have a different offset, producing a structurally different tree, and every query depending on those nodes would recompute even though the source code is semantically unchanged.

3.4.1.2 LANGUAGE SERVER INTEGRATION

Lowering produces, alongside the AST, a `SourceMap` that records the source span (byte range in the original input) of each AST path:

```
structure SourceMap where
  astToSpan : HashMap Path Span
```

The map is populated during lowering: when an AST node is constructed from a CST subtree, its AST path is associated with the span of that subtree, computed by accumulating token widths from the root. The language server uses this in two directions:

- *Hover*: given a cursor codepoint position, `astPathAtPosition` scans the entries to find the deepest AST path whose span contains the position. Hover content is keyed by AST path, so the subsequent lookup is direct.
- *Diagnostics*: the elaborator records diagnostics at AST paths. `spanForAstPath` looks up the recorded span, which the LSP layer converts into a UTF-16 range for the editor.

Source positions are materialised only at this final step. The entire elaboration pipeline operates on paths, never on byte offsets.

3.4.1.3 ERROR RECOVERY

On a parse error, a diagnostic is emitted and the parser skips to the next top-level declaration boundary (`def`, `inductive`, `axiom`, etc.). A single malformed declaration does not prevent the rest of the file from being elaborated.

3.4.2 Query-based elaboration

In a query-based elaborator, each phase of work is exposed as a query: parsing the source file, indexing its declarations, elaborating one declaration, fetching the elaborated form of a constant. Queries depend on other queries, with the dependency graph built at runtime. An edit invalidates the input queries that read from the edited file; the framework propagates invalidation through the graph until early cutoff stops the cascade.

The boundaries between queries fix the granularity at which cutoff can fire. Each boundary is a point at which the framework may detect that a recomputed value matches its cached predecessor; choosing the boundaries is the principal design decision.

3.4.2.1 Granularity

The output of elaborating a declaration is a kernel constant: a name, a type, and a body. We therefore make `constant` a query: the elaborated form of a named declaration is the unit at which the system caches and at which downstream queries fire cutoff. Smaller queries inside the elaboration of a single declaration (parsing, AST extraction, declaration indexing) are exposed as separate queries that compose into `elabDecl`. Larger granularities (per-file or per-module) would lose the ability to cut off at the boundary of an individual declaration; smaller granularities (per-expression) would add per-query overhead with no proportional reduction in recomputation, since conversion checking is the only mechanism by which one declaration influences another (Section 2.1.1).

3.4.2.2 The query chain

Our queries form a chain in which each query depends on a smaller prefix of work, terminating at file-content input queries:

$$\text{text} \longrightarrow \text{ast} \longrightarrow \text{declarationIndex} \longrightarrow \text{declAst} \longrightarrow \text{elabDecl} \longrightarrow \text{constant}.$$

`text` is the file's contents (an input); `ast`, `declarationIndex`, and `declAst` decompose parsing; `elabDecl` type-checks a declaration's subtree and fetches `constant` queries for any names it mentions; `constant` resolves a name to its elaborated form.

Cross-declaration dependencies enter the graph at `elabDecl`. When conversion checking unfolds a constant `foo` during the elaboration of `bar`, the framework registers an `elabDecl bar → constant foo` edge. Consider a file with three declarations:

```
def double (n : Nat) : Nat := Nat.mul 2 n
def quad (n : Nat) : Nat := double (double n)
def double_2 : double 2 = 4 := Eq.refl 4
```

The body of `quad` mentions `double`, so `elabDecl quad` fetches `constant double` while elaborating the application. The theorem `double_2` forces a conversion check between `double 2` and `4` while elaborating its proof; the check unfolds `double` and ι-reduces `Nat.mul`, registering a body-level dependency from `elabDecl double_2` on `constant double`.

Changing the body of `double` from `Nat.mul 2 n` to `Nat.add n n` leaves the type unchanged: `elabDecl quad`'s cache survives because it saw only the type, while `elabDecl double_2`'s cache is invalidated because it saw the body.

3.4.2.3 Dynamic dependencies

Whether an edge from `elabDecl bar` to `constant foo` appears in the graph depends on whether `bar`'s elaboration performs a conversion check that unfolds `foo`. This is not a property of the source code

alone: the conversion checker has speculative modes (`flex` and `rigid`) that may avoid the unfolding entirely, and the choice between them depends on the specific terms being compared. A static analysis cannot predict which dependencies will fire. The build system must observe them as the task runs, which requires a monadic task type and a scheduler that can resolve a fetch in the middle of a running task.

3.4.3 Bidirectional type checking

The elaborator follows the bidirectional discipline of Section 2.2, as two mutually recursive functions over the AST produced by the parser:

```
inferTm : TermContext n → Ast → OptionT (ElabM q₀) (Tm n × VTy n)
checkTm : TermContext n → VTy n → Ast → ElabM q₀ (Tm n)
```

`inferTm` synthesises a core term and its type from an AST node when no expected type is available. `checkTm` consumes an expected type as input and returns a core term at that type. Both are indexed by `TermContext n`, the local context after n bindings, and return a `Tm n` that can mention only those n indices. The AST split between the two functions is exhaustive: every constructor has exactly one applicable case in one mode.

3.4.3.1 Cases of inference

`inferTm` covers the AST constructors whose type is determined by the term alone.

Variables and constants. A local identifier is looked up in the context; the type is read out of the binding. A global identifier triggers `fetchConstant`, which issues a `Key.constant` query into the build system and records a dependency. The returned `Constant` carries its type, which is instantiated at the supplied universe arguments.

Universes. Type_ℓ infers to $\mathrm{Type}_{\ell+1}$. The level ℓ is parsed from the AST as a universe-level expression, not a term.

Applications. The head is inferred to a type, which must be a Pi after weak head normalisation. Because `inferTm` returns a `VTy` produced by `Ty.eval`, the type is already in WHNF; no extra `whnf` call is needed. The argument is checked against the domain, then the codomain is substituted by evaluating its closure in the extended environment.

Let-bindings with an annotation. The annotation provides the expected type; the bound value is checked, the body is inferred in the extended context.

Annotated terms $(e : A)$. The annotation switches direction: A is elaborated as a type, e is checked against A, and A is the inferred type.

The application case is the only one where a non-trivial value is constructed by hand:

```
| .node `Term.app #[f, a] ⇒ do
    let (fTm, fTy) ← inferTm ctx f
    let .pi _ aTy ⟨env, bTy⟩ := fTy
      | raiseError (.expectedFunctionType ctx.names (← fTy.quote))
    let aTm ← checkTm ctx aTy a
    let aVal ← aTm.eval ctx.env
    let bTyVal ← bTy.eval (env.cons aVal)
    return (.app fTm aTm, bTyVal)
```

The substitution $B[a/x]$ from the declarative Pi-elimination rule is implemented as `bTy.eval (env.cons aVal)`: evaluating the codomain's defunctionalised closure body in its captured environment extended by the argument's value. No explicit syntactic substitution is performed.

3.4.3.2 Cases of checking

`checkTm` covers the AST constructors whose elaboration depends on the expected type.

Lambdas without an annotation. The expected type must be a Pi; the parameter type A is read from it, and the body is checked against B in the context extended with $x : A$. If the lambda has a parameter annotation, the annotation is elaborated and compared against A by conversion; a mismatch is a type error.

Let-bindings without an annotation. The bound value is inferred; the body is checked at the expected type in the extended context.

Subsumption. When none of the checking cases applies, typically a variable, application, or annotated term in a checking position, `checkTm` falls back to inference and invokes the conversion checker to compare the inferred type against the expected type. This is the only point where conversion is invoked from `checkTm` itself; deeper conversion is done by the rules that drive it.

3.4.3.3 Error recovery

Three failure modes are handled without aborting the file. A literal `sorry` is parsed as an unspecified term; the elaborator emits an `Error.inferSorry` diagnostic and returns a fresh axiom of the expected type. An unbound variable emits `Error.unboundVariable` and returns a fresh axiom. A conversion failure during subsumption emits `Error.typeMismatch` carrying the inferred and expected types, and returns a fresh axiom at the expected type.

The fresh axiom is a `Constant.axiom` with a generated unique name and the appropriate type. Subsequent elaboration sees an inhabitant of the expected type and continues; downstream queries unfolding the axiom find no body and treat it as opaque. The `OptionT` layer on `inferTm` separates "no type could be inferred" (the option is `none`, the caller decides what to do) from "a hard error occurred" (raised through `ElabM`); only `checkTm`'s fallback path uses the option return.

`inferTm` and `checkTm` are mutually recursive with `checkIdent`, which looks up an identifier first in the local context and then in the global environment. `checkIdent` is where the cross-declaration dependency is registered: a global lookup that succeeds records a `Key.constant` dependency in the query graph, and the resolved `Constant` is cached in `ElabState.entryCache` to avoid repeated fetches within the same declaration's elaboration.

3.4.4 Normalisation by evaluation

The elaborator's NbE follows the algorithm of Section 2.3: evaluation maps syntax to values in WHNF, quotation reads them back, and defined constants evaluate to glued values whose bodies are forced on demand. This section fixes the data types and the `whnf` function that drive it.

The syntax `Tm`/`Ty` is the output of elaboration; the semantic domain `VTm`/`VTy` is what evaluation produces. Both are intrinsically scoped: `Tm n` and `VTm n` are indexed by the number of bound variables in scope, so a de Bruijn index or level can only refer to a binder the type system already knows about. Out-of-range references do not typecheck in Lean.

```
inductive Ty : Nat → Type
  | u  : Universe → Ty n
```

```
  | pi : Name → Ty n → Ty (n + 1) → Ty n
  | el : Tm n → Ty n

inductive Tm : Nat → Type
  | u'    : Universe → Tm n
  | var   : Idx n → Tm n
  | const : Name → List Universe → Tm n
  | lam   : Name → Ty n → Tm (n + 1) → Tm n
  | app   : Tm n → Tm n → Tm n
  | pi'   : Name → Tm n → Tm (n + 1) → Tm n
  | proj  : Nat → Tm n → Tm n
  | letE  : Name → Ty n → Tm n → Tm (n + 1) → Tm n

inductive VTy : Nat → Type
  | u  : Universe → VTy n
  | pi : Name → VTy n → ClosTy n → VTy n
  | el : Neutral n → VTy n

inductive VTm : Nat → Type
  | u'      : Universe → VTm n
  | neutral : Neutral n → VTm n
  | lam     : Name → VTy n → ClosTm n → VTm n
  | pi'     : Name → VTm n → ClosTm n → VTm n
  | glued   : Neutral n → Name → List Universe → VTm n

inductive Neutral : Nat → Type
  | mk {n} : Head n → Spine n → Neutral n
inductive Head : Nat → Type
  | var {n} : Lvl n → Head n
  | const {n} : Name → List Universe → Head n
inductive Spine : Nat → Type
  | nil {n} | app {n} : Spine n → VTm n → Spine n
  | proj {n} : Spine n → Nat → Spine n
inductive ClosTm : Nat → Type
  | mk {n m} : Env n m → Tm (m + 1) → ClosTm n
```

Three differences distinguish the two presentations. First, `Tm.var` uses a de Bruijn index `Idx n`, whereas a value can reference a variable only inside a `Neutral`, where the head is a level `Lvl n`. Second, application and projection do not appear as value constructors; an application of a blocked head accumulates in `Spine`, and the value's `lam` and `pi'` cases close over a `ClosTm`/`ClosTy` rather than carrying syntax under a binder. Third, the value-only constructor `VTm.glued` records a definition that has not yet been unfolded, holding the spine accumulated against the constant's name together with the universe arguments needed to instantiate the body once δ-reduction fires.

3.4.4.1 Weak head normalisation

`whnf` reduces a value until its outermost form is canonical or irreducible. Forcing a `VTm.glued` fetches the constant's body, evaluates it under the recorded universe substitution, re-applies the accumulated spine, and continues (δ-reduction). This is the only point at which a definition body is materialised. ι-reduction fires when a recursor is fully applied and its major premise is a constructor: the constructor's fields are substituted into the matching branch.

3.4.5 Conversion checking

Conversion is the hot loop of a dependent type checker. Every typing rule that crosses a definitional equality calls it, and most of those calls compare terms whose heads already agree. Forcing both sides to normal form just to discover this is wasteful. Kovács's smalltt [Kovács, 2023] structures conversion as a three-mode gamble that pays only for the unfoldings it actually needs.

The default mode is **rigid**. When two glued values present the same defined head, rigid bets the spines will agree without forcing either body, and hands control to **flex**. Flex walks the spines and unfolds nothing; the first mismatch fails immediately, and rigid then enters **full**, where every definition is unfolded on encounter. Once full takes over a subterm it stays in full for that subterm's recursion. There is at most one backtrack: the cheap flex pass either succeeds and saves the unfolding cost, or fails after walking only as much of the spines as matched.

Watching this on `Nat.add 2 3` against itself: both sides are glued at `Nat.add`, so rigid passes to flex. Flex walks the spines structurally — `succ (succ zero)` against the same for the first argument, then `succ (succ (succ zero))` against the same for the second — and reports agreement. `Nat.add`'s body is never fetched; no dependency on it is recorded.

Compare `Nat.add 2 3` against `5`. The heads differ — a glued definition on one side, a `Nat.succ` neutral on the other — so flex cannot apply. Full takes over, forces the left through `whnf`, which δ-unfolds `Nat.add` and fires the recursor's ι-rule three times to land on `succ (succ (succ (succ (succ zero))))`, which is `5`. The check succeeds, but `Nat.add`'s body was read, so the build framework records the dependency.

The checker also handles the two η rules of Section 2.1. Function η fires when one side of a comparison is a λ and the other is any term: both sides are applied to a fresh variable at the current de Bruijn level and their bodies are compared, so f is treated as a λ even when it is not syntactically one. Structure η fires when one side is a constructor application of a single-constructor inductive: each of the other side's projections is compared against the matching field, so s and $\mathsf{mk}(s.1, \ldots, s.k)$ are equated without forcing s into constructor form.

3.4.6 Incremental elaboration

Elaboration is decomposed into queries managed by a Shake-based build system. Each query has a tag and parameters; its result type is determined by the tag. `Key` has 20 cases; the principal ones are listed.

```
inductive Key where
  | astSourceMap : FilePath → Key
  | ast : FilePath → Key
  | declarationIndex : FilePath → Key
  | declAst : FilePath → Name → Key
  | declScope : FilePath → Name → Name → Key
  | elabDecl : FilePath → Name → Key
  | constant : FilePath → Name → Key
  | lookupInfo : FilePath → Name → Key
  -- ...
```

```
def Val : Key → Type
  | .astSourceMap _     ⇒ Ast × SourceMap × Array Diagnostic
  | .ast _              ⇒ Ast
  | .declarationIndex _ ⇒ HashMap Name Nat × Array Diagnostic
  | .declAst _ _        ⇒ Option Ast
  | .declScope _ _ _    ⇒ Bool
  | .elabDecl _ _       ⇒ Option Constant × ElabInfo
  | .constant _ _       ⇒ Option Constant
  | .lookupInfo _ _     ⇒ ElabInfo
  -- ...
```

When the elaborator encounters a constant reference, `fetchConstant` issues a `Key.constant` query, registering a dependency. An `entryCache` in `ElabState` memoises lookups within a single elaboration run to avoid repeated queries for the same name.

3.4.6.1 Query wiring

The `tasks` function maps each `Key` to its computation. Three representative cases illustrate how queries chain:

`constant filepath name` resolves a name to its elaborated form across files. It first tries the local file via `lookup filepath name`, which delegates to `elabDecl filepath name`. If that returns `none` (the name is not defined in this file), it fetches `transitiveImports filepath` to get the set of imported files, then tries `lookup path name` for each imported file until it finds a match. This two-phase lookup is the primary cross-file resolution mechanism: editing a definition in one file invalidates `constant` queries in other files that resolved to it.

`elabDecl filepath name` elaborates a single declaration. It fetches `declAst filepath name` to obtain the subtree for `name`, then runs the elaborator with the declaration-relative path stack initialised to the empty path. The diagnostics and hovers it produces are therefore independent of the declaration's absolute position in the file. The result is the elaborated `Constant` and diagnostic information.

The order check used during constant resolution ("may `name` be referenced from inside `currentDecl`'s elaboration?") is routed through `declScope filepath name currentDecl`. This query fetches `declarationIndex` and returns the `Bool` answer. Lifting the comparison out of `elabDecl`'s body means `elabDecl` does not depend on `declarationIndex` directly: insertions that shift indices recompute `declScope`, but its `Bool` result is unchanged for pre-existing pairs of names, so the cutoff fires at the `declScope` boundary and `elabDecl` is not re-elaborated.

`checkFile filepath` aggregates all diagnostics for a file. It fetches `astSourceMap filepath` for parse errors, `declarationIndex filepath` for the list of declarations, then `lookupInfo filepath name` for each declared name to collect elaboration diagnostics. Because each `lookupInfo` returns paths relative to its own declaration, `checkFile` prepends the declaration's current index at aggregation time, converting decl-relative paths to file-relative for the source-map resolver. This is the top-level query that the CLI and language server invoke.

3.4.6.2 The elaboration monad

`ElabM` combines incremental queries with elaboration state:

```
abbrev ElabM := ReaderT ElabContext (StateT ElabState (Task config q₀))
```

`Task` is the query monad: `Task.fetch` calls register dependencies. `StateT ElabState` threads the local environment, constant cache, and accumulated diagnostics and hovers. `ReaderT ElabContext` carries file path, declaration name, and universe parameters. q_0 identifies the current query in the dependency graph.

3.4.6.3 Query dependency graph

Figure 3 shows the query dependency graph for a two-definition file:

```
def foo : Type 1 := Type
def bar : Type 1 := foo
```

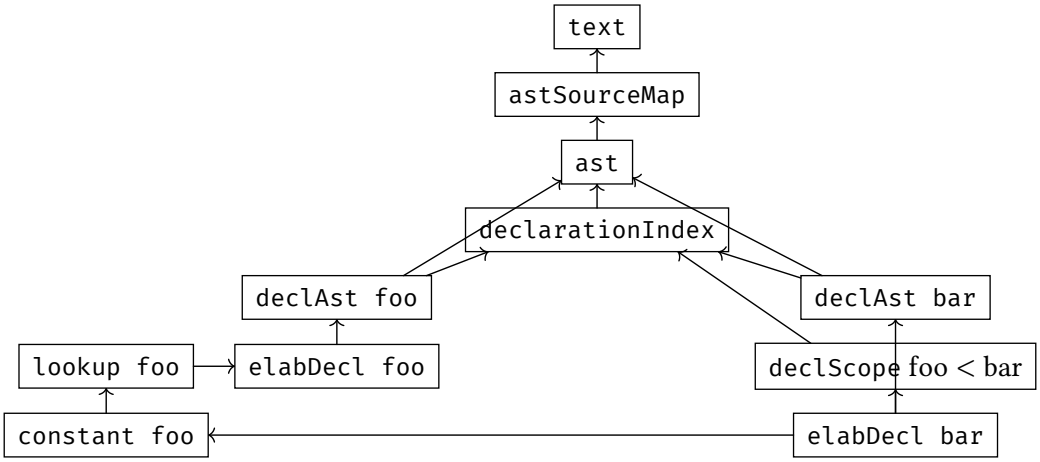

FIGURE 3: Query dependency graph for the two-definition example. Arrows go from each query to its dependencies. The horizontal edge from elabDecl bar to constant foo records that bar's elaboration fetched foo's body during conversion.

3.4.6.4 The Shake algorithm

The mechanism Shake follows when servicing a request for query q is Mitchell's verifying-trace scheme [Mitchell, 2012]. Each cached `Memo` stores not only q's value but a trace: the inputs the task read, the queries it fetched, and the hash of each at the time the task ran. On the next build, Shake checks whether the world still matches what the trace recorded, and runs the task again only when something has changed.

The check is recursive. Shake hashes every recorded input against the current input, and for every recorded query dependency it fetches that dependency — which may itself verify or recompute — and hashes the result. If all the hashes match, the cached value stands; if any one differs, the task runs, records fresh dependencies, and replaces the old `Memo`. The recursive structure is what makes verification *suspending* in the [Mokhov et al., 2020] classification: deciding whether q's memo is fresh suspends inside the decision for some dependency d, which suspends inside e, until every transitive trace entry has either matched its fingerprint or triggered a recompute.

Figure 4 traces an example. Starting from a two-declaration source `A.qdt`, the user inserts a new declaration `baz` between `foo` and `bar`. `text`, `ast`, and `declarationIndex` recompute to fresh hashes because the file structure changed. `declAst foo` and `declAst bar` see their input fingerprints (`ast`, `declarationIndex`) mismatch, so they too recompute—but `declAst` is keyed by paths relative to the declaration, so the recomputed subtrees are byte-identical to the previous ones and the new fingerprints match the stored ones. Cutoff fires here: `elabDecl foo`, `elabDecl bar`, and the `constant` queries above them see every input fingerprint match and serve cached values.

```
  def foo : Nat := 5
+ def baz : Nat := 7
  def bar : Nat := foo
```

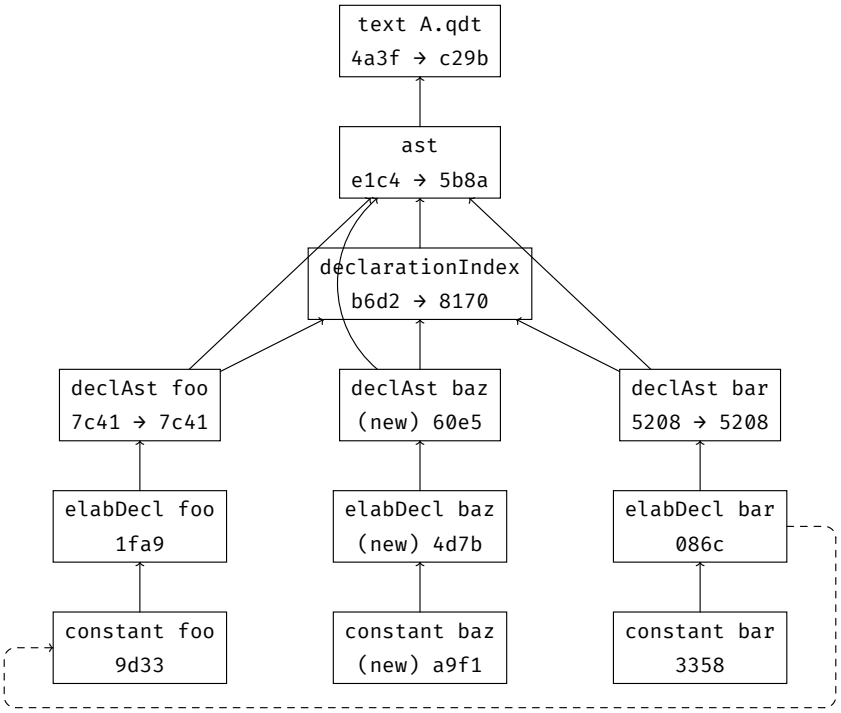

FIGURE 4: Rebuild trace after inserting `baz`. `h → h'` recomputed (cutoff when $h = h'$), `h` cache hit, `(new) h` first appearance. The dashed edge `elabDecl bar → constant foo` was recorded by conversion.

3.4.6.5 Early cutoff

Shake hashes each query's result alongside its dependency fingerprints. When a dependency is recomputed and its result hashes the same as before, the queries that depend on it are not recomputed; their fingerprints still match.

Consider appending a new definition to `Nat.qdt`. A new name appears, so the `declarationIndex` query is invalidated and the new definition's `elabDecl` query runs. But the existing `Nat.add`, `Nat.succ`, and other prior definitions produce the same `Constant` values as before. Their hashes are unchanged, so `constant Nat.add` returns the cached result. Files that import `Nat` see the same fingerprint and are not recomputed, even though `Nat.qdt` was edited.

Without early cutoff, any edit to `Nat.qdt` would cascade through every importing file. With early cutoff, the cascade stops at the first query whose result is unchanged.

3.5 Language server

The elaborator doubles as a language server, in the tradition of incremental attribute-grammar evaluation in syntax-directed editors [Demers et al., 1981] and the asynchronous-document model of Isabelle/PIDE [Wenzel, 2014]. Hover information and diagnostics are collected in `ElabState` during elaboration and served to a VSCode extension via the Language Server Protocol:

- **Diagnostics**: type errors, unbound variables, universe mismatches, positioned via path indices into the AST.

- **Hover**: the type under the cursor, or the full signature of a referenced constant.

3.6 Repository overview

The repository follows the standard structure of a Lean 4 project, with `lakefile.lean` at the root and a top-level `.lean` file re-exporting each module. The two principal modules are `Qdt/` (the elaborator and language server) and `Incremental/`, the build system framework, which is independent of the elaborator.

Directory	Files	Description	LoC
`Incremental/`	`Basic`, `FreeMonad`, `Busy`	Task abstraction, build configurations, fetch primitive	704
`Incremental/Shake/`	`Standard`, `StandardRdeps`, `Cancel`, `C`	Shake variants: Lean, rdeps-augmented Lean, effectful, and C versions	1,752
`Incremental/Test/`	`Fibonacci`, `Triangle`	End-to-end tests against concrete task graphs	128
`Incremental/c/`	`shake.c`, `salsa.c`	C implementations of Shake and Salsa	863
`Qdt/Theory/`	`Syntax`, `Universe`, `Global`	Core syntax, universe levels, global environment	758
`Qdt/Frontend/`	`Parser/Core`, `Desugar`	Parser to CST and lowering to AST with source map	1,390
`Qdt/`	`Bidirectional`, `Nbe`, `Conversion`	Bidirectional type checking, normalisation by evaluation, conversion checking	2,456
`Qdt/Incremental/`	`Query`, `Rules`	Query types and elaboration task rules	364
`Qdt/Test/`	`Universes`, `Trunc`, `Quotient`	Inline elaborator tests	714
`Qdt/Lsp/`	`Swap1`, `CrossFile`, `ImportCycle`	Scripted edit session tests	357
`FSWatch/`	`Manager`, `INotify`	File system watcher	387
`vscode-qdt/`	`src`, `syntaxes`	VS Code client	564
`examples/lean2-hott/`	`Lean2Export`, `port.sh`	Lean 2 HoTT library porter	469
`examples/`	`stdlib`, `long`	Example code and synthetic benchmarks	2,228
`bench/`	`cold`, `incremental`, `conversion`, `variants`	Benchmark drivers and conversion-checker variants	659
`.`	`Main`, `Lsp`, `lakefile`	CLI and LSP entry points; build config	356

4 Evaluation

This chapter asks three questions of the implementation. First (Section 4.1): what does the agreement theorem actually rule out, and what is left to behavioural testing? Second (Section 4.2): what does cold elaboration cost on the two corpora the elaborator was built against, and where in the pipeline does the time go? Third (Section 4.3): under nine edit categories chosen to isolate distinct framework mechanisms, what does an incremental rebuild cost, and what does that cost reveal about the framework underneath?

4.1 Correctness

The agreement theorem says: for any `Tasks` value and any inhabitant of `Build`, the inhabitant returns the same `Constant` as the reference `compute`. The theorem is unconditional in the executor's internal state and effect monads. What it does not say is that the `Tasks` value implements the type theory of Section 2.1.1 — that is, the proof rules out any drift between cache and compute, but it does not pin down what compute actually computes. The tests in Section 4.1.2, Section 4.1.3, and Section 4.1.4 close that gap.

4.1.1 Mechanised inhabitants

Table 1 lists the four mechanised inhabitants of `Build` and the axioms each depends on. `LessBusy` and `Shake` share one axiom budget: `propext` (propositional extensionality) and `Quot.sound` (soundness of quotient types), two ubiquitous axioms central to Lean 4's kernel. `ShakeStandardRdeps` additionally pulls in `Classical.choice` through `Std.Data.DHashMap`'s `get?_insert` lemmas. `Busy` depends on no axioms.

Inhabitant	Axioms	Behaviour
`Busy`	none	runs `compute` directly; agreement proof is `rfl`
`LessBusy`	`propext`, `Quot.sound`	intra-build memoisation; single-shot batch
`Shake`	`propext`, `Quot.sound`	persistent verifying traces; default executor
`ShakeStandardRdeps`	`propext`, `Classical.choice`, `Quot.sound`	reverse-dependency tracking on top of `Shake`'s cache; short-circuits the trace walk on clean queries

Table 1: Mechanised inhabitants of `Build`, with axiom dependencies from `AxiomCheck.lean`. `ShakeTrace` and `ShakeCancel` are `Shake` parameterised at different effect monads; their verification is `Shake`'s.

`Busy` is the reference inhabitant: its `build` field is `compute` with a single unfold, and the agreement proof is `rfl`. `LessBusy` is the first proof with content: an induction over the well-founded relation

on queries, showing the memo table agrees with `compute` on every key encountered. `Shake` extends this argument across persisted stores via the cross-input invariant from Section 3.2.4. `ShakeStandardRdeps` reuses `Shake`'s cross-input invariant per cache entry and bundles four structural invariants (`Sound`, `Complete`, `Closed`, `DownClosed`) on the rdeps-augmented `Persist` to license short-circuiting the trace walk; Section B walks the verification.

`ShakeTrace` and `ShakeCancel` reuse `Shake`'s proof. Each is `Shake` parameterised at a different effect monad (n = `TraceT` for the first; m = `Except Cancelled` for the second), with the same `tasks` value, the same store, and the same cache verification. The free theorem on `Tasks` lifts `Shake`'s agreement statement through the effect layer: a `MonadAction` between the layered monad and `Id` discharges the relatedness side-conditions on `pure` and `>>=`, and the conclusion specialises to agreement under the new effect. The verification is `Shake`'s; the layers contribute their `MonadAction` instances.

4.1.2 Elaborator tests

`Qdt/Test/` contains 14 files driven by the `#pass` and `#fail` macros. `#pass body` succeeds when `body` elaborates with no diagnostics; `#fail body with .error ..` succeeds when elaboration emits the named diagnostic. The current suite has 48 `#pass` and 37 `#fail` assertions, covering universes (`Universes.lean`, 41 cases), structure-η (`Eta.lean`), strict positivity (`Positivity.lean`), the recursor's ι-rule on `Eq` (`Eq3.lean`), quotient types (`Quotient.lean`), and the proof-irrelevant `Acc` accessor (`Acc.lean`). A representative `#fail`:

```
#fail (
  inductive Bad where
    | mk (f : Bad → Bad) : Bad
) with .nonPositiveOccurrence ..
```

The agreement theorem rules out drift between cache and specification; the suite rules out drift between specification and intent.

4.1.3 Language-server scenarios

`Qdt/Lsp/Test/` contains 17 scenarios driven by `Qdt/Lsp/Test.lean`. Each scenario sequences `setText` calls against an in-memory file system, fires a rebuild through the query layer, and asserts on diagnostics or hover responses produced by the language server. Categories: whitespace edits, body edits, type edits, cross-file propagation, import cycles, atomic moves, parser-error recovery, syntax recovery, and rename.

The `CrossFile.lean` scenario, where editing `foo` in `A.qdt` changes the hover for `bar` in `B.qdt`, runs:

```
setText (filepath := "A.qdt") qdt!( def foo := Type )
setText (filepath := "B.qdt") qdt!( import A
                                    def bar := foo )
hover (filepath := "B.qdt") ⟨2, 4⟩ "bar : Type 1" ⟨2, 4⟩ ⟨2, 7⟩

setText (filepath := "A.qdt") qdt!( def foo := Type 1 )
hover (filepath := "B.qdt") ⟨2, 4⟩ "bar : Type 2" ⟨2, 4⟩ ⟨2, 7⟩
```

Hover changes on the second `setText` without an explicit recomputation request. The cross-file dependency exists because `bar`'s elaborated type fetched `foo` during conversion in `A.qdt`; that fetch is the edge `Shake` invalidates.

4.1.4 Trace agreement

`ShakeTrace` records, per build, a forest of `DepNode (Key, durationNs, children)` with one node per query fetch. A fetch with `durationNs` above a 1 µs threshold corresponds to a task that ran; anything below is a cache hit. Across the 17 LSP scenarios, the recomputed set in the trace coincides with the invalidation set that `Shake.verifyDeps` walks (the proof's hit-or-miss decision), and the hit set coincides with the proof's cached set. The implementation walks the same dependency graph the proof reasons about: every fetch in the trace corresponds to an edge in the proof, and every edge in the proof appears as a fetch.

4.2 Cold performance

I evaluate cold elaboration on two corpora: the handwritten stdlib, and the `init/` subset of the Lean 2 HoTT library. The stdlib is a fairness check on the elaborator against code I wrote with its fragment in mind; the HoTT subset is a sanity check on a corpus I did not write, since a self-curated stdlib could understate the cost of features I happen to use sparingly.

Corpus	Files	Definitions	qdt cold (ms)
Handwritten stdlib	41	258	666
Lean 2 HoTT, init subset	23	1,092	11,770

Table 2: Cold elaboration time under `--build shake-c`, median of five trials per corpus.

4.2.1 The stdlib

The stdlib covers the fragments named in the proposal: Π and Σ types, `Nat`, `List`, `Eq`, recursors over a handful of indexed families, and supporting lemmas. Five-trial cold elaboration sits at 666 ms with trial-to-trial variation under 5%.

Phase breakdown. Table 3 attributes per-query durations from `--build shake-trace` to the top-level query that scheduled the work. The instrumented build is slower than `shake-c` (1.85 s versus 0.65 s wall), but per-call instrumentation overhead is approximately uniform across queries, so query-to-query proportions carry over.

Phase	Time (ms)	Share
Declaration elaboration (`elabDecl`)	1527	90.0%
Module-graph resolution (`transitiveImports`, `imports`)	114	6.7%
Per-file dispatch and indexing (`checkFile` shell, `declarationIndex`)	51	3.0%
Parsing (`ast`, `astSourceMap`)	6	0.4%

Table 3: Phase breakdown under `--build shake-trace`, summed across per-query durations from one trace. Declaration elaboration covers both bidirectional checking and conversion; no separate query distinguishes them.

Declaration elaboration absorbs nine-tenths of the work. The remaining tenth is split across module-graph traversal, per-file dispatch, and parsing in that order, with parsing essentially free. Where to look for cold-time gains is clear: the conversion checker and the NbE evaluator it calls. Nothing else is large enough to matter.

4.2.2 The Lean 2 HoTT init subset

The HoTT corpus is the `init/` directory of the Lean 2 HoTT library: 23 files, 1092 surface declarations, frozen at one commit. It is the largest slice that elaborates against the pre-HIT qdt; constructions in `hit/` and their downstream uses are out of scope. Cold elaboration takes 11.77 s on `shake-c`.

A detail of the source caught me out. Lean 2 inlines let-bodies before elaboration: a `let x := e in body` reaches the elaborator with every occurrence of `x` already substituted by e. A nested let-chain that reads as $O(n)$ in the source arrives as a term exponential in the nesting depth, with the same subexpression copied across thousands of leaves. A naive elaborator walks each copy independently and never finishes. Common-subexpression elimination on the elaborated terms is what makes this corpus elaborable at all — an unexciting optimisation forced by an unexpected feature of the source language.

4.3 Incremental re-elaboration

A rebuild should cost what its edit demands. To check whether ours does indeed do this, I designed nine edit categories so that each isolates one mechanism the framework relies on. Table 4 commits to a prediction per category before any measurement; Section 4.3.2 and Section 4.3.3 report what was measured.

4.3.1 Edit categories

The proposal asked for three edit kinds: local definition changes, type-signature modifications, and cascading dependency updates. These map onto leaf body, leaf type, and hub rename respectively. I added six more: a no-op so the verifier walks an unchanged cache, whitespace to separate the parser fingerprint from the AST fingerprint, an ill-typed body so the cache must survive a failed elaboration, a hub append for query creation, a delete-and-replace for key disappearance, and a parse error to exercise the cache while the file is unparseable. Each category is anchored at a fixed declaration: `Std/Nat.qdt` for hub-level edits, `Std/Ackermann.qdt` for leaf-level body edits, and `Std/Sigma.qdt` for whitespace and parse-error edits.

Category	Mechanism exercised	Predicted behaviour	Result
No-op	Trace verifier on every cached entry	All hits, zero fires; rebuild time is the verifier's per-entry cost times the cache size	held
Whitespace	Parser fingerprint vs. AST fingerprint	Parser fires once; AST cutoff stops the cascade at the leaf	held
Leaf body	Constant-hash early cutoff at the leaf	Body re-elaborates; dependents see unchanged hash and stop	held
Leaf body, ill-typed	Error propagation without cache corruption	Body fails elaboration; its `constant` entry is absent from the next cache; surviving entries are reused on the fix	held
Leaf type	Conversion-check cascades through dependents	Dependents re-check but do not re-elaborate; cost linear in the conversion-set of the type	held
Delete + replace leaf	Cache key invalidation on disappearance	Old key invalidates without orphaning; new key creates without spurious dependents	held
Hub append	New query in a heavily imported file	Single new query; existing constants in the same file remain cached	held
Hub rename	Mass invalidation across reverse-dependents	Cost scales with the reverse-dependency set; dependents re-check against the renamed constant rather than re-elaborating, so the cascade stays well below cold	held
Parse error	Cache survival across transient invalid states	Only the broken file invalidates; the next non-broken edit hits the cache from before the error	held

Table 4: Edit categories, the mechanism each one tests, the predicted behaviour, and the outcome confirmed in Section 4.3.2 and Section 4.3.3.

4.3.2 Per-file scatter

Figure 5 plots one point per (file, edit) pair on the stdlib for the three categories that apply to every file: noop, whitespace, and hub append. The horizontal axis is the cold elaboration time of the file's transitive closure (3–589 ms across the stdlib); the vertical axis is the incremental rebuild time. Each file is measured in a watch session whose entry module is the file itself, so cold and incremental walk the same dependency subtree — the parity diagonal compares like-for-like. Times come from `IO.monoNanosNow` and are reported in milliseconds with sub-millisecond precision.

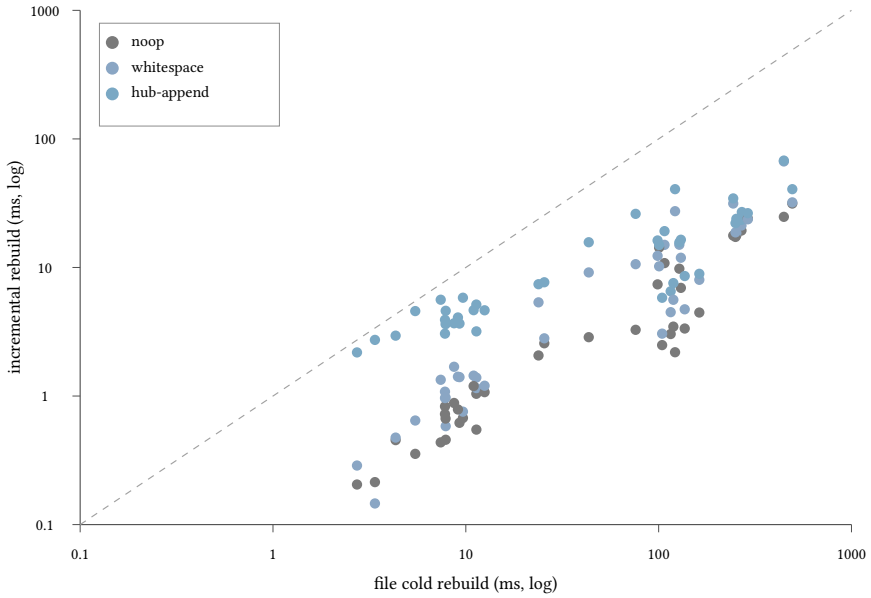

Figure 5: Per-file re-elaboration time against the file's transitive cold cost, stdlib under `shake-c`. One dot per (file, edit) across three universally applicable edit categories. Both axes are log-scale, and the diagonal is the parity line.

Every dot lies below the diagonal. At the extremes, the smallest file (3.2 ms cold), rebuilds in 0.17 ms on noop, 0.88 ms on whitespace, and 2.8 ms on hub append. the largest (589 ms cold), rebuilds in tens of milliseconds across the same three categories. Across the corpus the incremental cost grows much more slowly than the cold cost: about one order of magnitude separates the two at the top, two at the bottom. Hub append sits above noop and whitespace on every file, since a fresh declaration forces one new query through the build graph, while the other two categories ride entirely on cache verification.

4.3.3 Per-category timings

The scatter pools all three uniform categories across files. To compare categories one by one, I fix the corpus (the full stdlib, watched at module `Std`) and vary the edit. Table 5 reports five-trial medians per category in one watch session.

Category	Median (ms)	n	Errors
No-op	2.6	5	0
Whitespace	1.6	5	0
Leaf body	7.6	5	0
Leaf body, ill-typed	18.8	5	19
Leaf type	25.1	5	4
Delete + replace	30.0	5	0
Hub append	27.9	5	0
Parse error	3.5	5	1
Hub rename	76.0	1	92

Table 5: Per-category re-elaboration on `shake-rdeps`, stdlib watched at module `Std`. Five trials per category in one session, median reported. The `Errors` column gives the diagnostic count after the edit; hub rename is reported from a single trial because its mass invalidation does not reset cleanly on snapshot restore within a session.

The categories stratify across nearly two orders of magnitude. No-op and whitespace sit at the file-watcher floor (1.6–2.6 ms): no input changed except for a sub-AST detail, no query was dirtied, no work happened. Parse error joins them at 3.5 ms — the broken file's parser query invalidated, nothing downstream. Leaf body re-elaborates exactly one declaration at 7.6 ms; the ill-typed variant pays an additional 11 ms emitting nineteen diagnostics on downstream call sites. Leaf type, delete + replace, and hub append cluster between 25 and 30 ms, reflecting the size of each edit's reverse-dependency closure inside one file. Hub rename, the only edit invalidating a hub constant whose reverse-dependency closure spans 92 declarations, sits a factor of three above the next category at 76 ms — still 11 × below the 818 ms cold rebuild. Every category's cost reflects the edit's affected closure rather than the cache size, which is what Table 4 predicted of an rdep-tracked verifying-trace executor.

4.3.4 Two vignettes

To make the work pattern of a single edit visible, I trace two leaf-body changes to `ack_zero` in `Ackermann.qdt` against an 818 ms cold rebuild of the stdlib under `shake-rdeps`: one well-typed, one not. The first should leave the cache intact and recompute exactly the changed declaration; the second should report an error without corrupting the rest of the cache.

4.3.4.1 Leaf body, well-typed

Change `ack_zero`'s right-hand side from `Nat.succ n` to `ack 0 n`. Both terms type-check at the same type, since `ack 0 n` reduces to `Nat.succ n` by δ-unfolding `ack` and applying the recursor's ι-rule.

```
  def ack_zero (n : Nat) : ack 0 n = Nat.succ n :=
-   Eq.refl.{0} Nat (Nat.succ n)
+   Eq.refl.{0} Nat (ack 0 n)
```

No new diagnostic; hover on `ack_zero` unchanged.

Recomputed: `astSourceMap.qdt` `ast Ackermann.qdt` `declAst ack_zero` `elabDecl ack_zero` `constant ack_zero`.

Hit: `constant ack` `constant Nat.zero` `constant Nat.succ` `constant Eq.refl`, and every other entry in the file. 8 ms.

4.3.4.2 Leaf body, ill-typed

Replace `Nat.succ n` with `Nat.succ (Nat.succ n)` in `ack_zero`'s body. The right-hand side now has type `ack 0 n = Nat.succ (Nat.succ n)`, which does not match the declared signature.

```
  def ack_zero (n : Nat) : ack 0 n = Nat.succ n :=
-   Eq.refl.{0} Nat (Nat.succ n)
+   Eq.refl.{0} Nat (Nat.succ (Nat.succ n))
```

Type-mismatch on `ack_zero`'s body; nineteen further diagnostics from downstream `examples` reporting the dangling identifier.

Recomputed: `astSourceMap Ackermann.qdt` `ast Ackermann.qdt` `declAst ack_zero` `elabDecl ack_zero`, and each downstream `example`.

Hit: the cached `constant` entries for `ack`, `Nat.succ`, and `Eq.refl` survive; only the broken declaration's `constant` is missing from the next cache. 19 ms.

4.3.5 Comparison

The closest architectural neighbours are smalltt [Kovács, 2023], [Fredriksson, 2019], and Salsa [Matsakis and contributors, 2018]. smalltt optimises raw single-file throughput and does not address rebuild latency. [Fredriksson, 2019] exposes elaboration as queries against the Rock framework, the precedent our `Tasks` value resembles most. Salsa underlies rust-analyzer's incremental type-checking; our `salsa` inhabitant follows its discipline — request-driven, dirty-bit-on-fetch, no verifying trace — on the same `Tasks` value `shake` executes.

The inhabitants the chapter compares against are `shake-c` (a verifying-trace executor without rdep tracking), `salsa` and `salsa-c` (request-driven, dirty-bit-on-fetch, no verifying trace), and `shake-standard-rdeps` (rdep tracking with the structural agreement certificate). Table 6 reports cold and no-op for the five.

Inhabitant	Cold (ms)	No-op rebuild (ms)
`shake-rdeps`	818	2.6
`shake-standard-rdeps`	931	3.3
`salsa`	819	22.6
`salsa-c`	657	28.4
`shake-c`	666	54.3

Table 6: Median cold and no-op rebuild on the stdlib for five `Build` inhabitants. Cold is one-shot; no-op is the watch-mode rebuild after touching an unchanged file. Five-trial medians.

The factor-20 gap between `shake-rdeps` and `shake-c` on no-op is what rdep tracking buys. `shake-c` verifies its 124-entry cache against the inputs before consuming any cached value, walking the trace once per rebuild; that walk is most of the 54 ms total. `shake-rdeps` propagates input-fingerprint changes forward through stored reverse-dep edges and never inspects queries outside the affected closure; on a no-op, the closure is empty and the rebuild does almost nothing. `salsa` and `salsa-c` skip the verifier walk too but pay a different price: their agreement obligation is weaker, discharged by a different proof structure that this thesis does not develop. The verified `shake-standard-rdeps` (Section 3.3.5) matches `shake-rdeps`'s no-op latency to within a millisecond while keeping the structural agreement certificate, since its `set` short-circuit on input fingerprint equality returns the cached `Persist` unchanged and the four invariants transport across `Input.get j' = Input.get j` without re-elaboration.

4.3.6 Discussion

Under `shake-rdeps`, every category's cost reflects the edit's actual work: no-op and whitespace pay only the file-watcher floor, parse error invalidates one file's parser, leaf body re-elaborates exactly one declaration, leaf type and delete + replace and hub append do one file's reverse-dep cascade, and hub rename does the whole-corpus reverse-dep cascade — 1.6 ms to 76 ms across the nine categories. The four-tier stratification is the empirical content of Table 4: every category exercises one mechanism, and that mechanism's cost is now visible in the wall-clock time without a fixed verifier-walk overhead

masking it. The verifying-trace inhabitant `shake-c` would have hidden seven of the nine categories behind a 54 ms cache-verification floor (Table 6); rdep tracking lifts the floor and exposes the framework's per-edit behaviour as the chapter's hypotheses predicted.

5 Conclusion

5.1 Summary

All five desiderata are met and exceeded. Watch-mode rebuilds track the affected fragment on every edit category. Cold and cached elaboration produce the same elaborated forms across every elaborator and language-server test, the dynamic trace coincides with the agreement proof's hit-or-miss decision on every fetch, and the agreement itself is mechanised in Lean 4 against a reference batch semantics. The same `Tasks` value drives five inhabitants without source change or rerun of the proof; the effect-layer extensions inherit the theorem through a free theorem.

5.2 Lessons learnt

Reference counting and FBIP Lean's runtime mutates persistent data in place when the refcount is one, which keeps `StateM Cache` hashmap inserts at O(1) when the store is threaded linearly through the build. A stray `let` that retains a second reference silently degrades the write to O(n); the symptom is a slow benchmark, not a type error.

Verification forces design clarity More than one bug survived every test I wrote and only surfaced when a proof obligation refused to close. The proof is a sharper reader than I am; a `sorry` is a thumb on the scale.

Proof erasure The entire correctness invariant of `Build` — the `Value.spec` field, `WellFormed`, the `MonadAction.rel` axioms — lives in `Prop` and is erased at runtime. Co-locating proof with code costs nothing, which is why structural agreement is architecturally viable in Lean and would not be in a language whose type system cannot carry it.

5.3 Future work

Parallelism Static dependencies (parsing, independent files) parallelise under the applicative fragment of [Mokhov et al., 2020]'s task abstraction directly; conditional dependencies fit under *selective functors* [Mokhov et al., 2019] via speculative parallelism, sitting between applicative and monadic.

Type classes and coercions Resolution requires a metavariable layer for the unification variables in instance heads. Tabled resolution [Selsam et al., 2020] then treats each subgoal as a memoised query and its sub-subgoals as dependencies the framework already tracks [Saha and Ramakrishnan, 2003]; a resolver written through `pure`/`bind`/`fetch` inherits the parametricity certificate. How much of elaboration follows this pattern?

Tactics The natural extension to tactic languages would treat each goal state as a memoised query keyed by input state and tactic syntax, invalidating only the script suffix after an edit.

Bibliography

Abel, A., 2013. Normalization by Evaluation: Dependent Types and Impredicativity (Habilitation thesis).

Acar, U.A., Blelloch, G.E., Harper, R., 2002. Adaptive Functional Programming, in: Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL). ACM, pp. 247–259.. https://doi.org/10.1145/503272.503296

Boehm, B.W., 1988. A Spiral Model of Software Development and Enhancement. Computer 21, 61–72.. https://doi.org/10.1109/2.59

Bruijn, N.G. de, 1972. Lambda Calculus Notation with Nameless Dummies, a Tool for Automatic Formula Manipulation, with Application to the Church-Rosser Theorem. Indagationes Mathematicae (Proceedings) 75, 381–392.. https://doi.org/10.1016/1385-7258(72)90034-0

The Coq Development Team, 2024. The Coq Proof Assistant [WWW Document].. https://doi.org/10.5281/zenodo.14542673

Coquand, T., 1996. An Algorithm for Type-Checking Dependent Types. Science of Computer Programming 26, 167–177.. https://doi.org/10.1016/0167-6423(95)00021-6

de Moura, L., Ullrich, S., 2021. The Lean 4 Theorem Prover and Programming Language, in: Automated Deduction – CADE 28. Springer, pp. 625–635.. https://doi.org/10.1007/978-3-030-79876-5_37

Demers, A., Reps, T., Teitelbaum, T., 1981. Incremental Evaluation for Attribute Grammars with Application to Syntax-Directed Editors, in: Conference Record of the Eighth Annual ACM Symposium on Principles of Programming Languages (POPL). ACM, pp. 105–116.. https://doi.org/10.1145/567532.567544

Dunfield, J., Krishnaswami, N., 2021. Bidirectional Typing. ACM Computing Surveys 54, 98:1–98:38.. https://doi.org/10.1145/3450952

Dybjer, P., 1994. Inductive Families. Formal Aspects of Computing 6, 440–465.. https://doi.org/10.1007/BF01211308

Feldman, S.I., 1979. Make — A Program for Maintaining Computer Programs. Software: Practice and Experience 9, 255–265.. https://doi.org/10.1002/spe.4380090402

Fredriksson, O., 2019. sixty [WWW Document].. URL https://github.com/ollef/sixty

Goldfarb, W.D., 1981. The Undecidability of the Second-Order Unification Problem. Theoretical Computer Science 13, 225–230.. https://doi.org/10.1016/0304-3975(81)90040-2

Hammer, M.A., Phang, K.Y., Hicks, M., Foster, J.S., 2014. Adapton: Composable, Demand-Driven Incremental Computation, in: Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI). ACM, pp. 156–166.. https://doi.org/10.1145/2594291.2594324

Kladov, A., contributors, 2020. rust-analyzer [WWW Document].. URL https://rust-analyzer.github.io/

Kovács, A., 2024. Efficient Evaluation with Controlled Definition Unfolding [WWW Document].. URL https://andraskovacs.github.io/pdfs/wits24abstract.pdf

Kovács, A., 2023. smalltt [WWW Document].. URL https://github.com/AndrasKovacs/smalltt

The Lean 2 contributors, 2017. The Lean 2 Homotopy Type Theory Library.

Martin-Löf, P., 1984. Intuitionistic Type Theory. Bibliopolis, Naples.

The mathlib Community, 2020. The Lean Mathematical Library, in: Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020). ACM, New Orleans, LA, USA, pp. 367–381.. https://doi.org/10.1145/3372885.3373824

Matsakis, N., contributors, 2018. Salsa: A Generic Framework for On-Demand, Incrementalized Computation [WWW Document].. URL https://github.com/salsa-rs/salsa

Meertens, L.G.L.T., 1983. Incremental Polymorphic Type Checking in B, in: Conference Record of the Tenth Annual ACM Symposium on Principles of Programming Languages (POPL). ACM, pp. 265–275.. https://doi.org/10.1145/567067.567092

Microsoft, 2015. Roslyn: The .NET Compiler Platform.

Mitchell, N., 2012. Shake Before Building: Replacing Make with Haskell, in: Proceedings of the 17th ACM SIGPLAN International Conference on Functional Programming (ICFP). ACM, pp. 55–66.. https://doi.org/10.1145/2364527.2364538

Mokhov, A., Lukyanov, G., Marlow, S., Dimino, J., 2019. Selective Applicative Functors. Proceedings of the ACM on Programming Languages 3.. https://doi.org/10.1145/3341694

Mokhov, A., Mitchell, N., Peyton Jones, S., 2020. Build Systems à la Carte: Theory and Practice. Journal of Functional Programming 30, e11.. https://doi.org/10.1017/S0956796820000088

Norell, U., Chapman, J., 2009. Dependently Typed Programming in Agda, in: Advanced Functional Programming (AFP 2008), Lecture Notes in Computer Science. Springer, pp. 230–266.. https://doi.org/10.1007/978-3-642-04652-0_5

Pacak, A., Erdweg, S., Szabó, T., 2020. A Systematic Approach to Deriving Incremental Type Checkers. Proceedings of the ACM on Programming Languages 4, 127:1–127:28.. https://doi.org/10.1145/3428195

Pierce, B.C., Turner, D.N., 2000. Local Type Inference. ACM Transactions on Programming Languages and Systems 22, 1–44.. https://doi.org/10.1145/345099.345100

Porter, T.J., Kirisame, M., Wei, I., Panchekha, P., Omar, C., 2025. Incremental Bidirectional Typing via Order Maintenance. Proceedings of the ACM on Programming Languages 9.. https://doi.org/10.1145/3763117

Pratt, V.R., 1973. Top Down Operator Precedence (technical report). ACM.. https://doi.org/10.1145/512927.512931

Saha, D., Ramakrishnan, C.R., 2003. Incremental Evaluation of Tabled Logic Programs, in: Logic Programming (ICLP 2003), Lecture Notes in Computer Science. Springer, pp. 392–406.. https://doi.org/10.1007/978-3-540-24599-5_27

Selsam, D., Ullrich, S., de Moura, L., 2020. Tabled Typeclass Resolution [WWW Document].. URL https://arxiv.org/abs/2001.04301

Swierstra, W., 2008. Data Types à la Carte. Journal of Functional Programming 18, 423–436.. https://doi.org/10.1017/S0956796808006758

Ullrich, S.A., 2023. An Extensible Theorem Proving Frontend (Doctoral dissertation).

Voigtländer, J., 2009. Free Theorems Involving Type Constructor Classes, in: Proceedings of the 14th ACM SIGPLAN International Conference on Functional Programming (ICFP). ACM, pp. 173–184.. https://doi.org/10.1145/1596550.1596577

Wadler, P., 1989. Theorems for Free!, in: Proceedings of the 4th International Conference on Functional Programming Languages and Computer Architecture (FPCA). ACM, pp. 347–359.. https://doi.org/10.1145/99370.99404

Wenzel, M., 2014. Asynchronous User Interaction and Tool Integration in Isabelle/PIDE, in: Interactive Theorem Proving (ITP 2014), Lecture Notes in Computer Science. Springer, pp. 515–530.. https://doi.org/10.1007/978-3-319-08970-6_33

A Verification of Shake

This appendix walks through the Lean proof that the Shake cache is sound. It uses the build framework of 3.2: each query q has a result type, each input i has a value type, the family of tasks describes how to compute every query, and the reference semantics, called compute, is defined by well-founded recursion. Code is excerpted from `Incremental/Basic.lean`, `Incremental/FreeMonad.lean`, and `Incremental/Shake/Basic.lean`.

A.1 The proof obligation

Shake records what it observed during a build using two small record types. An *input dependency* is a pair (i, h) where i is the input that was read and h is the fingerprint of the value it returned. A *query dependency* is a triple (q, π, h): the fetched query, a proof that q precedes the current query in the well-founded order, and the fingerprint of the fetched result. Both records are parametric in a hash type H. Concrete builds instantiate H to 64-bit integers, but the verification only relies on injectivity of two hash embeddings: one for input values, written h_I, and one for query results, written h_R. The cache-miss step takes both as parameters.

A Shake *memo* for a query q bundles a computed value with arrays of these records and a universally quantified invariant (`Memo`):

```
structure Memo (q : 𝒞.Q) where
  value : 𝒞.R q
  inputDeps : Array (InputDepHash 𝒞.I H)
  queryDeps : Array (QueryDepHash 𝒞 q H)
  invariant :
    ∀ (ι : ∀ i, 𝒞.V i),
      (∀ p ∈ inputDeps, hI p.key (ι p.key) = p.hash) →
      (∀ p ∈ queryDeps, hR p.q (compute tasks ι p.q) = p.hash) →
      value = compute tasks ι q
```

The invariant promises that whenever every recorded input fingerprint agrees with ι and every recorded dependency fingerprint agrees with what compute returns under ι, the stored value equals the reference value at q under ι. Quantifying over ι is what lets a memo persist across builds: the same record stays valid under any input change that does not disturb the fingerprints.

The cache-miss step of Shake executes the task once under the current input function, call it ι_0, and packages the result into a fresh memo. The proof obligation is its invariant field, named `cacheMiss_invariant`: from observations made on a single run under ι_0, derive the universally quantified statement above. Discharging it has three ingredients: a parametricity interface for tasks (A.2), a free-monad reification of tasks under which a cross-input lemma is provable by induction (A.3), and an instrumented run that bridges the imperative recompute back to the reification (A.4). A.5 assembles them.

A.2 Parametric task interpretation

Each task is a parametric function: given any monad f and an interpretation of inputs and fetches in f, it produces a value in f. To reason about two different interpretations of the same task simultaneously, the formalisation uses the *monad actions* of 3.2. A monad action between two monads κ_1 and κ_2 is a function that, given any relation R between two (possibly distinct) types α and β, returns a relation between computations of type $\kappa_1\ \alpha$ and $\kappa_2\ \beta$. Write this lifted relation as R^*. It is required to satisfy two closure laws:

- Respect for pure: if $R(a, b)$ then $R^*(\text{pure}(a), \text{pure}(b))$.
- Respect for bind: if $R^*(m_1, m_2)$ holds, and whenever $R(a, b)$ the continuations satisfy $S^*(k_1(a), k_2(b))$, then $S^*(m_1 >>= k_1, m_2 >>= k_2)$.

A canonical example is a logical relation parameterised by a state: pair stateful computations whose return values are related, after running both from any starting state. The trace action of A.4 is one such; the FM-to-identity action of A.3 is another.

Every task carries a parametricity certificate (`Task.param`): for any monad action A and any two input families and two fetch families related pointwise by A at the equality relation, the two interpretations of the task itself are related by A at equality. The certificate is populated constructively by the four primitive task constructors. Choosing the action (deciding what "related" means) specialises this generic statement into the lemma needed at each step of the soundness proof.

A.3 Free-monad reification

Tasks are opaque parametric functions, so direct induction over them is impossible. To recover induction, each task is reified into a free monad whose constructors mirror the three primitive moves a task can make:

```
inductive FM (𝔊 : BuildConfig) (q₀ : 𝔊.Q) (α : Type) : Type
  | pure (a : α) : FM 𝔊 q₀ α
  | input (i : 𝔊.I) (k : 𝔊.V i → FM 𝔊 q₀ α) : FM 𝔊 q₀ α
  | fetch (q : 𝔊.Q) (hq : 𝔊.rel q q₀) (k : 𝔊.R q → FM 𝔊 q₀ α) : FM 𝔊 q₀ α
```

A tree is interpreted against an input function ι and a dependency oracle r. The function `evalTree` walks the tree, taking inputs from ι and fetches from r, and returns the value at the eventually reached leaf. Two companion functions, `evalTrace_inputs` and `evalTrace_deps`, return the lists of input and dependency entries visited *along the path the tree takes under the supplied oracles*. Both lists depend on ι and r: at an input node the continuation is fed the value $\iota(i)$, so different input functions can drive the walker into different subtrees and the recorded trace differs accordingly. Each input entry records the input index and the value read; each dependency entry records the query, its precedence proof, and the value returned by the oracle. An immediate consequence (lemma `evalTrace_deps_value`) is that, for every entry in the dependency trace, the recorded value is precisely what the oracle returned at that query.

The cross-input lemma `evalTree_cross` states that if a second pair of oracles agrees with the first at every entry the first pair caused to be recorded, then the tree evaluates to the same result under both pairs. The proof is by structural induction on the tree. At an input or fetch node, the head of the recorded trace pins down the value used at that step, forcing both evaluators to descend into the same continuation; the inductive hypothesis handles the rest.

Two further FM primitives package a single move as a tree returning its result: `pureInput` produces an input node whose continuation is `pure`, and `pureFetch` does the same for fetch. Using them, every task can itself be reified by interpreting it in the free monad: `tasksTree` is the tree obtained by running the task at q_0 in FM with these two primitives as the input and fetch interpretations.

```
def tasksTree (tasks : Tasks 𝒞) (q₀ : 𝒞.Q) : FM 𝒞 q₀ (𝒞.R q₀) :=
  (tasks q₀).fn (FM 𝒞 q₀) FM.pureInput FM.pureFetch
```

To relate this transcript to the reference semantics, define a monad action between FM and the identity monad whose underlying relation pairs a tree with a value when evaluating the tree produces that value. Closure under pure is immediate. Closure under bind uses an `evalTree_bind` equation showing that evaluating a tree then evaluating the continuation on the result equals evaluating the bound tree directly. Specialising the task's parametricity certificate at this action with the same ι and the same oracle on both sides yields the bridge lemma `tasksTree_eval_compute`:

```
theorem tasksTree_eval_compute (tasks : Tasks 𝒞) (q₀ : 𝒞.Q)
    (ι : ∀ i, 𝒞.V i) :
    FM.evalTree ι (compute tasks ι) (tasksTree 𝒞 tasks q₀) =
      compute tasks ι q₀
```

Composing this bridge with the cross-input lemma lifts the latter from FM to the reference semantics. The statement uses the trace of the syntactic transcript under the first input function:

```
theorem compute_cross (tasks : Tasks 𝒞) (q₀ : 𝒞.Q)
    (ι ι' : ∀ i, 𝒞.V i)
    (hin : ∀ p ∈ FM.evalTrace_inputs ι (compute tasks ι)
        (tasksTree 𝒞 tasks q₀), ι' p.i = p.v)
    (hdep : ∀ p ∈ FM.evalTrace_deps ι (compute tasks ι)
        (tasksTree 𝒞 tasks q₀),
      compute tasks ι' p.q = p.r) :
    compute tasks ι q₀ = compute tasks ι' q₀
```

Two input functions that agree at the trace entries produced by the first compute to the same value at q_0.

A.4 Trace action

The cross-input lemma above reasons about trees, but Shake runs the task in a state-transformer monad that accumulates two arrays as the task executes. The recording is asymmetric. Each input read pushes a fresh hashed input entry onto the input array, unconditionally: reading an input is cheap, and verifying a duplicate at cache-hit time costs nothing. Each fetched dependency, by contrast, is pushed onto the dependency array *only if no entry with the same query is already there.* At cache-hit time the dependency array is replayed to recursively verify each fetched query, and a duplicate would force two recursive verifications of the same dep.

The deduplicating push is a small operation, `dedupPush`: given a new entry and an accumulator, return the accumulator unchanged if it already contains an entry with the same query, and append the new entry otherwise. Folding `dedupPush` over a list of dependency entries (hashing each value along the way) gives the operation `pushAll`, which produces the final array Shake stores.

Deduplication discards information, but the soundness of recording only one representative per query is captured by `pushAll_complete`. The statement: if every entry in the input list satisfies that hashing its recorded value yields a fixed function of its recorded query (the "target" function from queries to hashes), then for every input entry the result array contains a witness entry with the same query and the same target hash. Concretely, after deduplication every distinct query has exactly one representative, and that representative carries the correct hash. The cache-miss proof instantiates the target with the function sending each query q to h_R applied to q and to compute under ι_0 at q.

The bridge from the imperative run to the syntactic transcript is the monad action `traceAction`. Given an imperative computation and a tree, the relation says that on any successful return from any starting state, three things hold simultaneously:

- The produced value relates (under the underlying relation) to the value obtained by evaluating the tree at ι_0 and compute under ι_0.
- The final input array is the initial input array, extended by the hashed input trace of the tree at the same oracles.
- The final dependency array is the initial dependency array, extended via `pushAll` by the hashed dependency trace.

"Successful return" is captured by a `CanReturn` predicate: an imperative computation can return some final state-and-value pair when running it from a given starting state may yield that pair. For deterministic monads like the identity or pure state monads this is just functional equality of the run with `pure`; for richer monads (exceptions, IO) it picks out the genuinely possible returns.

Two atomic-task lemmas, `runInput'_rel` and `runFetch'_rel`, verify that the imperative primitives Shake uses for each move satisfy the trace-action relation against the syntactic primitives. The first says: running the imperative "read input i" primitive pushes the single hashed entry for i at $\iota_0(i)$ onto the input array and matches the trace of `pureInput`. The second says: running the imperative "fetch q" primitive (after consulting the cache to obtain a verified value v) pushes the deduplicated entry for q at v onto the dependency array and matches the trace of `pureFetch`. Each imperative primitive is a single state update and the corresponding tree primitive walks exactly one node, so both proofs are immediate.

Specialising the task's parametricity certificate at the trace action, with these two lemmas as the input and fetch hypotheses, yields a single statement: the full imperative run of the task at q_0 is trace-action-related to the full syntactic transcript `tasksTree`.

A.5 Cache-miss invariant

Starting the imperative run from the empty initial state and instantiating the trace-action relation at the actual return gives three concrete observations about the final state: the value produced, the input array `ins`, and the dependency array `deps`. The first conjunct of the relation, combined with `tasksTree_eval_compute`, gives that the value equals compute at q_0 under ι_0. The second conjunct, with empty initial input array, says that `ins` is the input trace under ι_0 and compute under ι_0, hashed entry-wise and converted to an array. The third, with empty initial dependency array, says that `deps` is the result of folding `pushAll` over the dependency trace from the empty array.

These three observations are precisely the hypotheses of the named lemma:

```
theorem cacheMiss_invariant {ι₀ : ∀ i, 𝔊.V i} {q₀ : 𝔊.Q}
    {value : 𝔊.R q₀}
    {ins : Array (InputDepHash 𝔊.I H)}
    {deps : Array (QueryDepHash 𝔊 q₀ H)}
    (hval : value = compute tasks ι₀ q₀)
    (hin_trace : ins =
      ((FM.evalTrace_inputs ι₀ (compute tasks ι₀) (tasksTree 𝔊 tasks q₀)).map
        fun p ⇒ ⟨⟨p.i⟩, hI p.i p.v⟩).toArray)
    (hdep_trace : deps =
      pushAll hR (FM.evalTrace_deps ι₀ (compute tasks ι₀) (tasksTree 𝔊 tasks
q₀))
        (#[] : Array (QueryDepHash 𝔊 q₀ H))) :
    ∀ (ι : ∀ i, 𝔊.V i),
      (∀ p ∈ ins, hI p.key (ι p.key) = p.hash) →
      (∀ p ∈ deps, hR p.q (compute tasks ι p.q) = p.hash) →
      value = compute tasks ι q₀
```

Fix an arbitrary ι together with the input-hash agreement and dependency-hash agreement assumed by the invariant. Below, an input entry's input index and recorded value are written `i` and `v`; a dependency entry's query, precedence proof, and recorded value are written `q`, `π`, and `r`; and the hash field on either kind of entry is written `hash`. The proof has three steps.

1. *Input promotion.* Take an entry of the input trace under ι_0 with index `i` and recorded value `v`. By the second observation, the stored input array contains the hashed pair $(\mathtt{i}, h_I(\mathtt{i}, \mathtt{v}))$. The input-hash agreement at this stored entry equates $h_I(\mathtt{i}, \iota(\mathtt{i}))$ with $h_I(\mathtt{i}, \mathtt{v})$. Injectivity of h_I at `i` strips the embedding, leaving $\iota(\mathtt{i}) = \mathtt{v}$. So ι and ι_0 agree at every entry of the input trace.
2. *Dependency promotion.* Take an entry of the dependency trace under ι_0 and compute under ι_0, with query `q` and recorded value `r`. By `evalTrace_deps_value`, the recorded value `r` is compute at `q` under ι_0. Applying `pushAll_complete` with the target function sending each q' to h_R of q' and compute at q' under ι_0 produces a witness in the stored dependency array whose query is `q` and whose hash is the target value $h_R(\mathtt{q}, \text{compute under } \iota_0 \text{ at } \mathtt{q})$. The dependency-hash agreement at the witness, applied to the same witness, equates $h_R(\mathtt{q}, \text{compute under } \iota \text{ at } \mathtt{q})$ with $h_R(\mathtt{q}, \text{compute under } \iota_0 \text{ at } \mathtt{q})$. Injectivity of h_R at `q` strips this to compute at `q` under ι equals compute at `q` under ι_0, which equals `r`. So compute under ι and compute under ι_0 agree at every entry of the dependency trace.
3. *Closing.* The cross-input lemma `compute_cross`, applied with the two agreements just established, gives compute at q_0 under ι_0 equals compute at q_0 under ι. Composing with the first observation (that the value equals compute at q_0 under ι_0) proves the value equals compute at q_0 under ι, as required.

Plugging this lemma into the cache-miss branch of Shake's run discharges the invariant field of the freshly built memo.

B Verification of reverse-dependency optimisation

This appendix walks through the verification of `ShakeStandardRdeps`, the rdeps-augmented Shake inhabitant of Section 3.3.5. It reuses the cache-miss invariant of Section A and adds the structural invariants that let `set` and `build` short-circuit. The following code is from `Incremental/Shake/StandardRdeps.lean`.

B.1 Persistent state and invariants

The persistent state extends Shake's `Cache` with two reverse-dependency arrays and a working set of dirty queries:

```
structure Persist where
  cache      : DHashMap 𝒞.Q (Memo hI hR tasks)
  queryRdeps : Array (𝒞.Q × 𝒞.Q)
  inputRdeps : Array (𝒞.I × 𝒞.Q)
  dirty      : Array 𝒞.Q
```

A pair $(d, q) \in$ `queryRdeps` records that q's last recompute fetched d; a pair $(i, q) \in$ `inputRdeps` records that q read input i. Four invariants over a `Persist` p together form `WellFormed ιₒ p`:

```
def Sound (ιₒ : ∀ i, 𝒞.V i) (p : Persist) : Prop :=
  ∀ q mm, p.cache.get? q = some mm → q ∉ p.dirty →
    (∀ d ∈ mm.inputDeps, hI d.key (ιₒ d.key) = d.hash) ∧
    (∀ d ∈ mm.queryDeps, hR d.q (compute tasks ιₒ d.q) = d.hash)

def Complete (p : Persist) : Prop :=
  (∀ q mm, p.cache.get? q = some mm → ∀ d ∈ mm.inputDeps, (d.key, q) ∈
p.inputRdeps) ∧
  (∀ q mm, p.cache.get? q = some mm → ∀ d ∈ mm.queryDeps, (d.q,   q) ∈
p.queryRdeps)

def Closed (p : Persist) : Prop :=
  ∀ q mm, p.cache.get? q = some mm → ∀ d ∈ mm.queryDeps, ∃ mm', p.cache.get?
d.q = some mm'

def DownClosed (p : Persist) : Prop :=
  ∀ q mm, p.cache.get? q = some mm → q ∉ p.dirty →
    ∀ d ∈ mm.queryDeps, d.q ∉ p.dirty

structure WellFormed (ιₒ : ∀ i, 𝒞.V i) (p : Persist) : Prop where
  sound      : Sound ιₒ p
  complete   : Complete p
  closed     : Closed p
  downClosed : DownClosed p
```

- *Sound*: a clean cached entry's recorded hashes match the live inputs and dependency values. Composed with `mm.invariant` (the universally quantified field of `Memo`, proved at cache-miss time by `cacheMiss_invariant`), it gives `value = compute tasks ι₀ q` on the spot.
- *Complete*: every dependency edge of every cached memo is registered in the rdeps arrays. Invalidating an input reaches every affected cached query through the rdeps graph.
- *Closed*: the cache is downward closed under the query-dependency relation: a cached memo never points at an absent dep.
- *DownClosed*: cleanliness is downward closed under the same relation: a clean memo's deps are clean.

The `Build`'s state is the subtype that carries `WellFormed` as an irrelevant proof:

```
σ := { sp : J × Persist // WellFormed (Input.get sp.1) sp.2 }
```

Each operation produces a fresh subtype value; the proof obligation discharged at each step is preservation of the four invariants.

B.2 Reverse-dependency closure

`invalidate` extends the dirty set by the transitive rdeps closure of an input change:

```
def invalidate (p : Persist) (i : 𝒞.I) : Persist :=
  { p with dirty := p.dirty ++ closure p.queryRdeps (seedOf p.inputRdeps i) }
```

`seedOf p.inputRdeps i` extracts the queries that have i as a recorded input dependency. `closure` performs a bounded breadth-first walk over the `queryRdeps` edge relation starting from this seed.

Cycles in `queryRdeps` mean structural termination cannot see why the walk terminates. The recursion is instead made well-founded on a lexicographic measure with a static upper bound on the visited set's size:

```
def closureGo (rdeps : Array (𝒞.Q × 𝒞.Q)) (bound : Nat) (worklist : List 𝒞.Q)
    (visited : Array 𝒞.Q) (hv : visited.size ≤ bound) : Array 𝒞.Q :=
  match worklist with
  | []      ⇒ visited
  | k :: ws ⇒
    if visited.toList.contains k then closureGo rdeps bound ws visited hv
    else if h : visited.size < bound then
      let new := rdeps.foldl (init := []) fun acc (p, c) ⇒
        if p = k then c :: acc else acc
      closureGo rdeps bound (new ++ ws) (visited.push k) (by simp; omega)
    else visited
termination_by (bound - visited.size, ws.length)
```

- *Push branch.* `visited.size` grows; the measure drops in its first component.
- *Skip branch.* `worklist` shortens; the second component drops while the first stays equal.

The top-level `closure` instantiates `bound` at $|\text{seed}| + |\text{rdeps}| + 1$. Both characterising lemmas thread two additional invariants through the recursion: `visited.toList` is `Nodup`, and its elements lie in the ambient set `seed.toList ++ rdeps.toList.map (·.snd)`. The push branch preserves these via two auxiliary lemmas (`push_toList_nodup` adds k to the nodup list because k was not already

there, `push_toList_inUniv` adds k because k came off the worklist which is itself contained in the ambient set).

The `else visited` branch is dead: if `visited.size = bound`, then by `List.Subperm.length_le` applied to the `Nodup` list against the ambient set, `visited.toList.length` is at most the ambient set's length, which is $|\text{seed}| + |\text{rdeps}| = \text{bound} - 1$, contradicting `visited.toList.length = bound`. `visited_size_lt` packages this contradiction.

Two lemmas characterise the result:

- `closureGo_worklist_subset` says every member of the worklist is in the result. Proof by structural induction over the worklist; recursive calls remain in range because of the `Nodup`/`Subperm` argument above.
- `closureGo_step_closed_aux` says the result is closed under the rdeps edge: if k is in the result and (k, q') is in `rdeps`, then q' is too. The proof carries a stronger invariant through the induction: every visited k has its rdeps targets either already visited or still in the worklist, and shows each step preserves it.

Specialising to `closure`:

```
theorem closure_step_closed (rdeps : Array (𝒞.Q × 𝒞.Q)) (seed : Array 𝒞.Q)
    (k : 𝒞.Q) (hk : k ∈ closure rdeps seed)
    (q' : 𝒞.Q) (he : (k, q') ∈ rdeps) :
    q' ∈ closure rdeps seed
```

Together with `closure_seed_subset` (every seed element ends up in the closure), it discharges `invalidate`'s preservation lemmas.

B.3 Preservation under set

`set i v` dispatches on whether the input value's hash changed:

- *Unchanged hash.* Injectivity of h_I at i gives that the input function is pointwise unchanged. The `Persist` is reused verbatim, and `WellFormed` transports across `Input.get j' = Input.get j` by □.
- *Changed hash.* `set i v` produces `invalidate p i`. `Complete` and `Closed` ignore the dirty set and carry through unchanged; the two non-trivial preservation lemmas are:

```
theorem invalidate_preserves_downClosed
    (p : Persist) (i : 𝒞.I) (hC : Complete p) (hD : DownClosed p) :
    DownClosed (invalidate p i)

theorem invalidate_preserves_sound
    (p : Persist) (ιₒ : ∀ i, 𝒞.V i) (i : 𝒞.I) (v : 𝒞.V i)
    (hS : Sound ιₒ p) (hC : Complete p) (hCl : Closed p) (hD : DownClosed p) :
    Sound (Function.update ιₒ i v) (invalidate p i)
```

DownClosed under invalidate. Take a clean cached entry (q, m) after invalidation and a queryDep d of m. The post-invalidation dirty set is `p.dirty ++ closure` …, and cleanliness of q means q is in neither summand:

- From the original `DownClosed`, $d.\mathsf{q} \notin$ `p.dirty`.

- If d.q were in `closure`, then by `closure_step_closed` applied to the edge $(d.q, q) \in$ `queryRdeps` (supplied by `Complete`), so would q — contradicting cleanliness.

So d.q is in neither summand and stays clean.

Sound under invalidate. By well-founded induction on `𝒞.rel`. The conclusion to prove at q for a clean cached memo `mm` is that every recorded input fingerprint matches h_I at the updated ι_0, and every recorded dep fingerprint matches h_R at `compute` under the updated ι_0.

- *Input case.* By `Complete`, $(d$.`key`, $q) \in$ `inputRdeps`. If d.`key` $= i$ then q would be in the closure seed and hence dirty, contradiction; otherwise the input is unchanged at d.`key` and the old hash agreement applies.
- *Dep case* at a queryDep d of `mm`. The old `Sound` at q (legal because $q \notin$ `p.dirty`, since q is clean after invalidation, which only added to `dirty`) gives `hR d.q (compute tasks ι₀ d.q) = d.hash`. By `Closed`, d.q has a cached memo `mm′`; by the original `DownClosed` at q, d.q was clean before invalidation; by `closure_step_closed`, d.q is not in the closure either, so d.q stays clean after invalidation. The IH at d.q therefore gives that `mm′` is `Sound` under the updated ι_0. Apply `mm′.invariant` twice, once at the old ι_0 with the agreements from the old `Sound` at d.q, giving `mm′.value = compute tasks ι₀ d.q`, and once at the updated ι_0 with the agreements from the IH, giving `mm′.value = compute tasks (updated ι₀) d.q`. Composing, the two `compute`s at d.q agree. Rewriting the old `Sound`'s dep agreement at d by this equality yields the required agreement under the updated input.

`wellFormed_invalidate` packages the four conclusions: `Complete` and `Closed` carry through unchanged, `DownClosed` and `Sound` use the preservation lemmas above.

B.4 Preservation under build

`populate q` is defined by `𝒞.wf.fix` on `𝒞.rel`, with body `populateBody`. The result type at each query is

```
structure PopulateResult (q : 𝒞.Q) (pIn : Persist) where
  persist        : Persist
  value          : Value tasks ι₀ q
  hWF            : WellFormed ι₀ persist
  hNotDirty      : q ∉ persist.dirty
  memo           : Memo hI hR tasks q
  hCache         : persist.cache.get? q = some memo
  hValEq         : memo.value = value.val
  hCacheMono     : ∀ q' mm', pIn.cache.get? q' = some mm' →
                     ∃ mm'', persist.cache.get? q' = some mm''
  hDirtyAntiMono : ∀ q', q' ∈ persist.dirty → q' ∈ pIn.dirty
```

so a successful populate carries: the updated `Persist` with its `WellFormed` proof, a verified `Value`, the inserted memo and its location in the cache, and two monotonicity witnesses (the new cache extends the old, the new dirty set is contained in the old). `populateBody` dispatches on whether the cached entry can be trusted:

Fast path. If q is not in `p.dirty` and `p.cache.get? q = some mm`, then `Sound.value`

```
def Sound.value (hS : Sound ι₀ p) :
    ∀ q mm, p.cache.get? q = some mm → q ∉ p.dirty →
```

```
      mm.value = compute tasks ι₀ q :=
  fun q mm hG hN ⇒
    let ⟨hI', hR'⟩ := hS q mm hG hN
    mm.invariant ι₀ hI' hR'
```

strips `mm.invariant` against the live hash agreements and returns the cached value with its agreement certificate. The `Persist` is unchanged, so `WellFormed` carries through verbatim.

Recompute path. `populateRecompute` calls `runWalk`, a function that walks the free-monad reification `tasksTree 𝔖 tasks q` (from Section A) interpreting each `FM.input` against the live ι_0, each `FM.fetch q'` against a recursive `populate q'`, and recording the trace as it goes. Bind its result to r. `runWalk` returns

```
structure RunWalkResult (t : FM 𝔖 q₀ α) (pIn : Persist)
    (accQueryDeps : Array (QueryDepHash 𝔖 q₀ H)) where
  value          : α
  persist        : Persist
  hWF            : WellFormed ι₀ persist
  inputDepsList  : List (InputDepHash 𝔖.I H)
  queryDepsArr   : Array (QueryDepHash 𝔖 q₀ H)
  hValue         : value = FM.evalTree ι₀ (compute tasks ι₀) t
  hInputDeps     : inputDepsList = (FM.evalTrace_inputs ι₀ (compute tasks ι₀)
t).map
                                     (fun e ⇒ ⟨⟨e.i⟩, hI e.i e.v⟩)
  hQueryDeps     : queryDepsArr  = pushAll hR
                                     (FM.evalTrace_deps ι₀ (compute tasks ι₀)
t)
                                     accQueryDeps
  hDepsInCache   : ∀ d ∈ FM.evalTrace_deps ι₀ (compute tasks ι₀) t,
                     ∃ mm, persist.cache.get? d.q = some mm
  hDepsNotDirty  : ∀ d ∈ FM.evalTrace_deps ι₀ (compute tasks ι₀) t,
                     d.q ∉ persist.dirty
  hCacheMono     : ∀ q' mm', pIn.cache.get? q' = some mm' →
                     ∃ mm'', persist.cache.get? q' = some mm''
  hDirtyAntiMono : ∀ q', q' ∈ persist.dirty → q' ∈ pIn.dirty
```

The three load-bearing fields are `hDepsInCache`, `hDepsNotDirty`, and the two monotonicity witnesses. Each `FM.fetch q` in the trace produced a `PopulateResult` at q; its `hCache` and `hNotDirty` witness that q is cached and clean in the local persist after the child call. As the rest of `runWalk` runs further child calls, those witnesses survive thanks to the recursive `hCacheMono`/`hDirtyAntiMono`.

A fresh `Memo` is built from the trace, with `cacheMiss_invariant` (from Section A) discharging its universally quantified `invariant` field. The new cache is `r.persist.cache.insert q memo`, and the new dependency edges are added by `recordRdeps`:

```
def recordRdeps (q : 𝔖.Q) (mm : Memo q) (p : Persist) : Persist :=
  let p := mm.inputDeps.foldl (init := p) fun p d ⇒
    { p with inputRdeps := p.inputRdeps.push (d.key, q) }
  mm.queryDeps.foldl (init := p) fun p d ⇒
    { p with queryRdeps := p.queryRdeps.push (d.q, q) }
```

Alongside, the dirty set drops q via filter. The four `WellFormed` invariants on the resulting `Persist` are re-established by four preservation lemmas, all sharing one dispatch helper for reasoning about lookup-after-insert:

```
private theorem cache_insert_cases
    {p : Persist} {q q' : 𝒬.Q} {memo : Memo q} {mm' : Memo q'}
    (hG : (p.cache.insert q memo).get? q' = some mm') :
    (∃ h : q = q', mm' = h ▸ memo) ∨ (q ≠ q' ∧ p.cache.get? q' = some mm')
```

Proof: rewrite with `DHashMap.get?_insert` and dispatch on `(q == q') = true`. In the `true` branch, `LawfulBEq` strips the boolean and `cases hG` substitutes `mm' = memo`; in the `false` branch, the lookup falls through to the underlying cache and $q \neq q'$ comes from `beq_self_eq_true`.

The four preservation arms:

- `recordRdeps_preserves_sound`. On the inserted side ($q' = q$), apply `cacheMiss_verify_ins` and `cacheMiss_verify_deps`: each of these states that the freshly built `memo` has hash agreements at the trace, derived from the same trace observations Shake uses in Section A. On the inherited side ($q' \neq q$), the dirty set has shrunk only by filtering out q, so $q' \notin$ filtered implies $q' \notin$ original, and the old `Sound` applies.
- `recordRdeps_preserves_complete`. On the inserted side, two helpers `inputPass_inputRdeps_adds` and `queryPass_queryRdeps_adds` witness that the freshly pushed rdeps edges (d, q) for each $d \in$ `memo.inputDeps`/`memo.queryDeps` are present. On the inherited side, two mono lemmas `foldl_inputRdeps_mono`/`foldl_queryRdeps_mono` carry the existing edges across the two pushes.
- `Closed` on the inserted entry: `hDepsInCache` from `runWalk` gives, for every dep d in the trace, that $d.q$ is cached. The trace deps and `memo.queryDeps` differ only by the dedup-and-push pass; `pushAll_origin` matches each element of the output array to a trace dep with the same `.q`, so the cached witness lifts. A dual helper `cache_insert_get?_exists` then lifts the cached witness from the pre-insert cache to the inserted one.
- `DownClosed` on the inserted entry runs the same lift on `hDepsNotDirty` instead of `hDepsInCache`. On existing entries, filter-monotonicity preserves not-dirty.

`populate q` returns a `PopulateResult` whose `hWF` field is the new `WellFormed` and whose `value` field carries the agreement certificate `value = compute tasks ι₀ q`.

B.5 Commentary

Two ideas carry the proof:

- `DownClosed` is what makes the recursion legal. The fast path of `populateBody` needs each dep clean for `Sound.value` to apply, and the well-founded induction in `invalidate_preserves_sound` needs each dep clean to invoke the IH at it.
- The Shake trick of universally quantifying `mm.invariant` over the input function carries through into the rdeps induction. Applying it twice at the same memo, once at the old ι_0, once at the updated ι_0, bridges the two `compute`s at a dep without revisiting Shake's cross-input lemma.

`Complete`, `recordRdeps_preserves_complete`, and the push-membership lemmas are mechanical: they record graph edges and replay them. The closure is a standard BFS with a static bound, with the wrinkle that the bound argument lets the `else visited` branch be dead rather than a graceful

early exit. `cache_insert_cases` saves four near-identical proofs about lookup-after-insert from being written four times.

C Project Proposal

Introduction and Description

This project aims to implement an incremental, query-based elaborator for a dependently typed toy language, for efficient type checking in interactive development environments, such as proof assistants. Traditional type checkers for dependent types operate in batch mode, re-checking entire programs after each change. This project will investigate query-based compilation techniques, building on Olle Fredriksson's work and modern language servers, to enable efficient incremental type checking.

The core objective is to build an elaborator that can efficiently respond to local changes in source code by recomputing only the affected parts of the type checking process. This will be achieved using a query-based architecture where type checking operations are decomposed into memoisable queries with explicit dependencies. The implementation will use the Salsa framework, written in Rust, which provides the tools for incremental on-demand computation by automatically tracking dependencies and memoisation.

Starting Point

No prior work was done before October.

Substance and Structure

1. Core Language: a minimal dependently typed calculus with Pi, universes, and basic inductive types. The syntax will support function definitions, pattern matching, and type annotations.
2. Query-based elaborator: bidirectional type checking decomposed into queries (e.g., `infer_type`, `check_type`, `normalise_term`). Each query can be memoised, with proper dependency tracking to enable incremental recomputation.
3. Incremental infrastructure: use of the Salsa framework to handle source file changes, manage parse trees, and dependency graphs for queries. Detection of changes and lazy memoisation strategies. This is the most important component of the project.
4. Performance evaluation framework: benchmarking suite to measure incremental performance against batch re-elaboration, including synthetic workloads simulating typical editing patterns.

Key algorithms include type unification, bidirectional type checking with constraint generation, normalisation for dependent types.

Success Criteria and Evaluation Plan

Success Criteria

1. Correctly type check a language with dependent types, including Pi types, Sigma types and inductive types
2. Demonstrate measurable performance improvement for incremental changes vs full re-elaboration

Evaluation Plan

- Correctness: test suite of programs including simple proofs of equality, existential and universal quantification (dependent pair and arrow types).
- Performance and scalability: Benchmarks measuring re-elaboration time for:
  - local definition changes
  - type signature modifications
  - cascading dependency updates

  on sample programs, generated by a computer, of at least 1000 lines of code

Work Plan

Michaelmas Term

- 9 Oct – 22 Oct: Read Salsa documentation, study dependent type implementation, implement toy examples, design language syntax and semantics
- 23 Oct – 6 Nov: Implement elaborator for dependent types over a query-based interface
- 7 Nov – 19 Nov: Complete elaborator for dependent types, including identity types, dependent pairs; write example programs
- 20 Nov – 3 Dec: Decompose elaborator, and other stages of compilation process into fine-grained queries

Lent Term

- 22 Jan – 4 Feb: Implement incrementalisation with change detection, cache invalidation, and dependency tracking, utilising Salsa
- 5 Feb – 18 Feb: Complete incrementalisation, with source-based tracking
- 19 Feb – 4 Mar: Optimise incrementalisation, query granularity and dependency tracking
- 5 Mar – 18 Mar: Evaluation and performance benchmarks, with synthetic programs

Easter Term

- 30 Apr – 15 May: Dissertation

Resources Declaration

- Personal laptop (32 GB RAM, Intel Ultra 7 155) for development
- Backup strategy: Git repository on GitHub
- Contingency plan: Backup laptop available

Ethics Approval

Not required as no human participants or personal data involved.